SUP'COM

المـــدرســـة العليـــا للمـــواصـــلات بتونـــس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Cycle de formation des ingénieurs en Télécommunications

Option :

MIT

Rapport de Projet de fin d'études

Thème :

« Conception et développement d'une plateforme CI/CD pour les applications web multi-services »

Réalisé par :

Ayoub BERGAOUI

Encadrants :

Mme. Sameh NAJEH

M. Aymen GHEZAIEL

Travail proposé et réalisé en collaboration avec :

3S Standard Sharing Software

Année universitaire : 2019-2020

Ecole Supérieure des Communications de Tunis

SUP'COM

2083 Cité Technologique des Communications - Elghazala - Ariana – Tunisie

Tél. +216 70 240 900 – site web : www.supcom.mincom.

RÉSUMÉ :

L'objectif de ce projet est de mettre en place une plateforme d'intégration continue et de livraison et déploiement continu pour les applications web multi-services. Après une étude des principaux concepts et pratiques de DevOps, nous avons effectué une étude comparative des outils de ces pratiques. Suite à cette étude comparative, une analyse des besoins et une conception de l'architecture du projet se sont apparues nécessaires, avant l'implémentation de ce pipeline CI/CD. Ce projet a permis de minimiser le temps de production et d'organiser le cycle de vie de chaque application. En effet, une application web doit passer par son répertoire de gestion du code source, passant par un serveur d'intégration continue et un outil de contrôle qualité jusqu'à son déploiement automatique. Cette dernière phase, basée sur la technologie de virtualisation en conteneurs, transforme ce code source en programme en exécution.

MOTS CLÉS : DevOps, Intégration continue, Déploiement continu, Livraison continue, Automatisation, Virtualisation, Docker, Cloud Computing, Orchestration Conteneurs, …

ABSTRACT:

The objective of this project is to set up a continuous integration platform and continuous delivery and deployment for multi-service web applications. After a study of the main concepts and practices of DevOps, we did out a comparative study of the tools of these practices. Following this comparative study, a needs analysis and a design of the project architecture appeared necessary, before the implementation of this CI / CD pipeline and platform. This project helped us to minimize production time of applications and organize the life cycle of each one. Indeed, now in this project, a web application must go through its source code management directory, passing through a continuous integration server and a quality control tool until its automatic deployment. This last phase of this project, based on container virtualization technology, transforms this source code into a program in execution.

KEY WORDS: DevOps, Continuous integration, Continuous deployment, Continuous delivery, Automation, Virtualization, Docker, Cloud Computing, Container orchestration, …

REMERCIEMENTS

Il m’est offert ici par quelques lignes la possibilité de remercier les personnes qui ont contribué à faire de ces quatre mois passés un bon stage au sein du groupe 3s.

Je remercie tout particulièrement Monsieur Aymen Ghezaiel, chef de l’équipe système à 3S et mon encadrant professionnel, de m’avoir donné l’opportunité d’effectuer ce stage, ainsi que toute l’équipe pour leur encadrement et le suivi apporté.

Je tiens à remercier infiniment Madame Sameh Najeh, de m’avoir encadré et conseillé tout au long de ce stage.

Je voudrais remercier également mes parents et toute ma famille pour leur soutien inconditionnel.

À toutes ces personnes, je souhaite encore une fois leur exprimer toute ma gratitude pour m’avoir permis de réaliser mon stage dans les meilleures conditions durant ces derniers mois.

TABLE DES MATIERES

INTRODUCTION GENERALE 10
CHAPITRE 1 : CONTEXTE GENERAL DU PROJET 11
1. Introduction 12
2. Présentation générale de Standard Sharing Software (3S) 12
2.1. Historique 12
2.2. Activités 12
2.3. Services 12
2.4. Organigramme 13
2.5. Filiales 3S: 13
2.6. Partenaires 13
3. Étude de l'existant 14
3.1. Analyse de l'existant (les outils actuels) 14
3.2. Limites de l'existant 15
3.3. Solution proposée 16
4. Architecture générale de la solution 17
4.1. Git 17
4.2. Jenkins 18
4.3. Sonar 18
4.4. Nexus 19
4.5. Ansible 19
4.6. Docker 20
4.7. Kubernetes 21
5. Conclusion 21
CHAPITRE 2 : ETAT DE L'ART ET CONCEPT DE DEVOPS 22
1. Introduction 23
2. Intégration continue 23
2.1. Définition 23
2.2. Étapes de l'intégration continue (CI) 23
2.2.1. Étape 1 : Commettre le code source 24
2.2.2. Étape 2 : Récupération du code 25
2.2.3. Étape 3 : Compilation/Exécution des tests 25
2.2.4. Étape 4 : Mesure de qualité 26
2.2.5. Étape 5 : Création de l'artéfact 26

2.3. Composants de la CI 27
3. Déploiement continu /Livraison continue (CD) 29
3.1. Définition 29
3.2. Déploiement continu Vs Livraison continue 29
3.3. Étapes du CD 29
3.3.1. Commettre son code 30
3.3.2. Détection d'un changement 30
3.3.3. Tirer son code 30
3.3.4. Push vers DockerHub 30
3.3.5. Serveur d'automatisation continue le déploiement 30
3.3.6. Déploiement vers les serveurs cibles 30
3.4. Avantages du CD 31
4. Cloud computing – AWS 31
4.1. Définition 31
4.2. AWS & Services utilisés dans le projet 32
4.2.1. Service EC2 32
4.2.2. AMI 32
4.2.3. Adresses IP Élastiques 32
4.2.4. Service RDS 32
5. Virtualisation 33
5.1. Définition 33
5.2. Domaines de virtualisation 33
5.3. Différentes techniques de virtualisation 34
5.4. Technique de virtualisation 38
6. Conclusion 39
CHAPITRE 3 : ANALYSE ET CONCEPTION UML 40
1. Introduction 41
2. Analyse des besoins 41
3. UML – Diagrammes des cas d'utilisations 42
3.1. Cas d'utilisation pour le site web 44
4. UML – Diagramme des composants 45
5. UML – Diagramme de déploiement 46
6. Conclusion 46
CHAPITRE 4 : REALISATION 47
1. Introduction 48

2. Environnement de travail 48
3. Implémentation 48
3.1. Mettre en place le 1er élément du pipeline ~ Jenkins 48
3.2. Intégration d'un serveur Tomcat dans le pipeline 50
3.2.1. Déploiement d'un fichier .war dans le serveur Tomcat en créant une tâche Jenkins (Option 1) 50
3.2.2. Automatisation du déploiement (PollSCM) 51
3.3. Intégration de SonarQube dans le pipeline 52
3.4. Partie déploiement automatisé – Ansible 55
3.5. Intégration de DockerHub dans le pipeline 56
3.5.1. Tagging l'image docker en utilisant Ansible playbook 57
3.5.2. Version finale - Job Jenkins 59
3.6. Intégration de Kubernetes dans le pipeline 60
3.6.1. Mettre en place le premier nœud 61
3.6.2. Mettre en place le deuxième nœud 61
3.6.3. Création des fichiers YAML de déploiement et service 61
3.6.4. Intégration de Kubernetes avec Ansible 63
3.6.5. CI Job dans Jenkins pour créer une image Docker 66
3.6.6. CD Job dans Jenkins pour déployer dans Kubernetes 67
3.7. Interfaces web 68
4. Conclusion 72
CONCLUSION GENERALE 73
ANNEXES 74
LISTE DES RÉFÉRENCES 78

LISTE DES FIGURES

Figure 1 : Organigramme de 3S 13
Figure 2 : Filiales de 3S 13
Figure 3 : Logos des partenaires de 3S 13
Figure 4 : Avant DevOps 14
Figure 5 : Association Dev et Ops 15
Figure 6 : Solution DevOps 16
Figure 7 : Architecture générale de la solution 17
Figure 8 : Logos de Git, Github et GitLab 17
Figure 9 : Logo de Jenkins 18
Figure 10 : Logo de SonarQube 19
Figure 11 : Logo de Nexus Sonatype 19
Figure 12 : Logo d'Ansible 20
Figure 13 : Logo de Docker 20
Figure 14 : Logo de Kubernetes 21
Figure 15 : Concept général de la CI 23
Figure 16 : Architecture générale de la CI 24
Figure 17 : Étape 1 de la CI 24
Figure 18 : Étape 2 de la CI 25
Figure 19 : Étape 3 de la CI 25
Figure 20 : Étape 4 de la CI 26
Figure 21 : Étape 5 de la CI 27
Figure 22 : Composants de la CI 27
Figure 23 : Différence entre CD et cd 29
Figure 24 : Exemple de CD 30
Figure 25 : Logo AWS 32
Figure 25-b : Services AWS 32
Figure 25-c : Virtualisation 33
Figure 26 : Diagramme des cas d'utilisations du pipeline 42
Figure 27 : Diagramme des cas d'utilisations du site web 44
Figure 28 : Diagramme des composants du pipeline 45
Figure 30 : Page d'accueil Jenkins 48
Figure 31 : 1er Job Jenkins 49
Figure 32 : Commande du 1er job 49
Figure 33 : Test de fonctionnement du 1er job 49
Figure 35 : Authentification avec Jenkins 50
Figure 36 : Action après la construction 51
Figure 37 : schéma : /opt/tomcat/ 51
Figure 38 : Configuration du PollSCM 51
Figure 39 : Push vers GitHub 52
Figure 40 : Job lancé automatiquement 52
Figure 41-b : Créer un utilisateur et une BD pour Sonar 52
Figure 41-c : Configuration sonar scanner dans Jenkins 53
Figure 41-d : Clé d'authentification entre Sonar et Jenkins 53
Figure 42 : Résultat de la première analyse sonar coté Jenkins 53
Figure 42-b : Résultat de la première analyse coté sonar 54
Figure 42-c : Suggestion de correction Sonar 54
Figure 43 : Résultat du deuxième analyse Sonar coté Jenkins 54
Figure 43-b : Résultat du deuxième analyse coté Sonar 55
Figure 44 : Test Ping 55
Figure 45 : Publish over SSH 56
Figure 46 : Push l'image vers Docker Hub 56
Figure 46-b : Répertoire Docker Hub 56

Figure 47: Playbook 1 (Docker) 57
Figure 47-b : Exécution Playbook 1 (Docker) 57
Figure 48 : Playbook 2 (Docker) 58
Figure 48-b : Exécution Playbook 2 (Docker) 58
Figure 49 : Fichier Hosts 58
Figure 50 : Caractéristiques du job final (Docker) 59
Figure 51 : Test final 59
Figure 52 : Installation de kubeadm, kubelet, kubectl, kubernetes-cni 60
Figure 53: kubeadm init 60
Figure 54 : Afficher les nœuds avec kubectl 60
Figure 55 : Mettre en place le premier nœud 61
Figure 56 : Mettre en place le deuxième nœud 61
Figure 58 : Script YAML de service 63
Figure 59 : Vérification après exécution de deploy et service 63
Figure 60: /opt/Kubernetes/ 64
Figure 61 : fichier Hosts 64
Figure 62 : Playbook “deployment” 64
Figure 63: Playbook “service” 65
Figure 64 : Exécution playbooks 65
Figure 65 : CI job K8S 66
Figure 65-b : Résultat CI job 66
Figure 75-c : CD job s’exécute automatiquement 67
Figure 66 : CD Job K8S 67
Figure 66-b : Résultat CD job 67
Figure 67 : Page d’accueil 68
Figure 68 : Page d’accès aux serveurs 69
Figure 69 : Page d’exécution des « jobs » 70
Figure 70 : Page d’accès aux scripts 71
Figure 71 : Page contact 72

LISTE DES TABLEAUX

Tableau 1 : Différentes techniques de virtualisation 34

Tableau 3 : Comparaison d'outils CI 36

Tableau 4 : Comparaison d'outils gestion de dépôt binaires 37

Tableau 5 : Comparaison d'outils contrôle qualité 37

Tableau 6 : Comparaison d'outils de configuration automatique 38

Tableau 8 : Explication des cas d'utilisations (1) 43

Tableau 9 : Explication des cas d'utilisations (2) 44

INTRODUCTION GENERALE

L'arrivée de l'entreprise numérique implique que les méthodes de création d'applications doivent constamment être adaptées, à un rythme toujours plus accru. Il n'est plus possible d'attendre 6 mois pour la mise en production d'un produit. D'ailleurs, il doit non seulement pouvoir être mis sur le marché plus rapidement, mais de nouvelles fonctions doivent pouvoir être ajoutées sans aucune difficulté, et les bugs découverts doivent pouvoir être corrigés.

Avec l'évolution des projets dans des configurations logicielles et d'infrastructure de plus en plus complexes et avec ces contraintes de temps au marché, faire intervenir autant d'équipes peut mener à un mur de conflits. Des difficultés apparaissent en outre souvent entre l'équipe de développement et l'équipe opérationnelle. En effet, lorsque les développeurs pensent à additionner une nouvelle fonction et donc évoluer leurs applications, en contrepartie, l'équipe opérationnel cherche à stabiliser le système informatique avant tout. Ainsi, chacun poursuit ses processus, et utilise ses propres outils qui sont souvent rarement communicants. Ces oppositions engendrent des retards de livraison, des dépenses supplémentaires pour l'entreprise qui n'ont pas été prévu initialement, avec un impact sur le client dont sa satisfaction est le point primordial des préoccupations de l'entreprise.

Tout cela mène à une approche DevOps. Cette approche permet de palier ces problèmes. Cependant, l'équipe de développeurs n'est pas la seule à devoir réagir de manière rapide et efficace mais aussi l'équipe opérationnelle, qui doit déployer et surveiller les nouvelles applications, doit elle-même aussi pouvoir le faire. C'est dans ce contexte que s'inscrit le présent projet au sein de Standard Sharing Software (3S). Il s'agit de faire la conception et le développement d'une plateforme d'un pipeline d'intégration continue et de déploiement continu (CI/CD) pour les applications web. Ce rapport est composé de 4 chapitres :

- Dans le premier chapitre, nous présentons l'entreprise accueillante ainsi que le cadre général du projet.
- Le deuxième chapitre permet de présenter les principaux concepts de base de ce projet, suivi d'une étude comparative des outils DevOps les plus répandus.
- Le troisième chapitre est consacré à l'analyse et la conception UML et la mise en lumière sur le déroulement de l'étape de la réalisation.
- Dans le quatrième chapitre, nous abordons la partie réalisation et les tâches dans l'implémentation de notre projet.

CHAPITRE 1 : CONTEXTE GENERAL DU PROJET

CHAPITRE 1 : CONTEXTE GENERAL DU PROJET

1. Introduction :

Dans ce chapitre, nous mettons en place le cadre général du projet en présentant l'entreprise accueillante. Ensuite, nous enchainerons avec une analyse de l'existant dans l'entreprise afin de tirer les limites puis une solution et son architecture générale à mettre en place par la suite.

2. Présentation générale de Standard Sharing Software (3S) :

3s est un intégrateur d'infrastructures et de services à forte valeur technologique, qui offre des solutions de la technologie de l'information de bout en bout. C'est une société tunisienne à vocation technologique aux nombreuses filiales située à Montplaisir, Tunis.

Elle est spécialisée également dans la formation aux nouvelles technologies, les réseaux mobiles, le développement des logiciels de transaction, les codes-barres, les centres d'appel et le commerce électronique. [1]

2.1. Historique :

3S a créé sa filiale Global Net en 1997, qui est devenue un fournisseur de services Internet. En 2007, la société est devenue le premier partenaire tunisien de Cisco Systèmes. [1]

2.2. Activités :

Le groupe 3S compte trois activités principales : L'intégration des réseaux, systèmes et l'édition de logiciels.

2.3. Services :

3S intervient au niveau du cycle de vie d'un projet. Elle maîtrise l'intégralité des infrastructures réseau jusqu'aux applications, avec pour ambition de concrétiser les objectifs business et d'optimiser les performances de l'entreprise cliente. Nous citons quelques services de 3S tels que l'intégration et déploiement de solutions réseaux, système et de de sécurité informatique, aussi bien le support technique de ces solutions après la mise en place. Ainsi, 3S offre l'assistance au démarrage et des formations.

2.4. Organigramme :

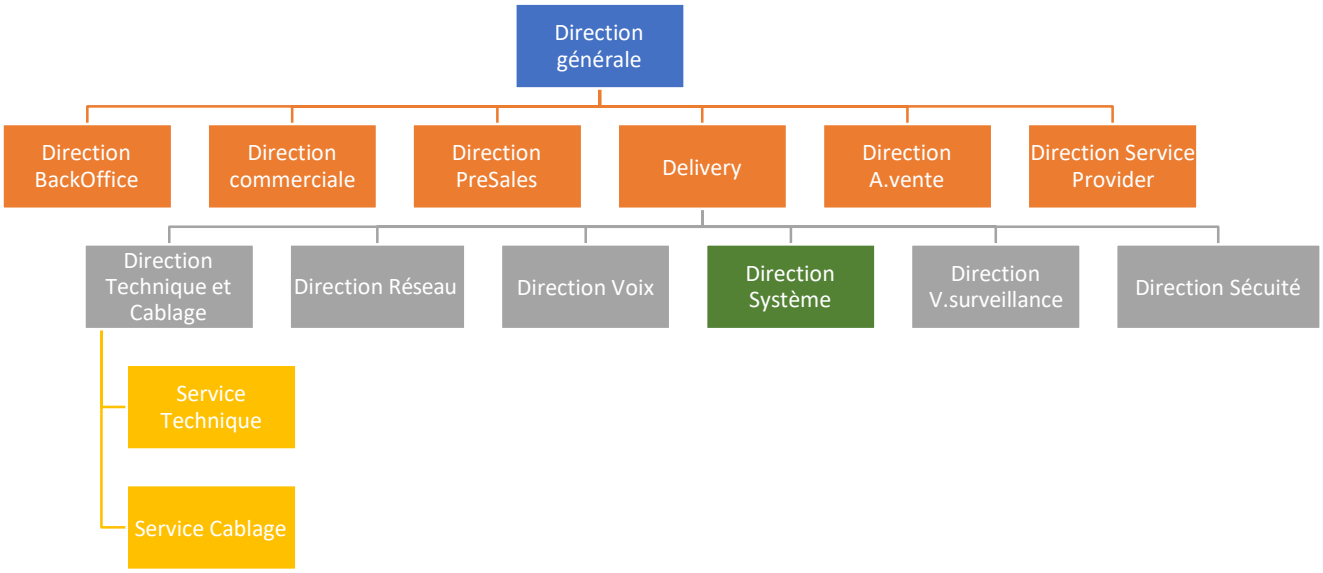


Figure 1 : Organigramme de 3S

2.5. Filiales 3S:



Figure 2 : Filiales de 3S

2.6. Partenaires :

Concrétisé par des alliances stratégiques conclues avec des leaders mondiaux sur leurs segments de marché comme :

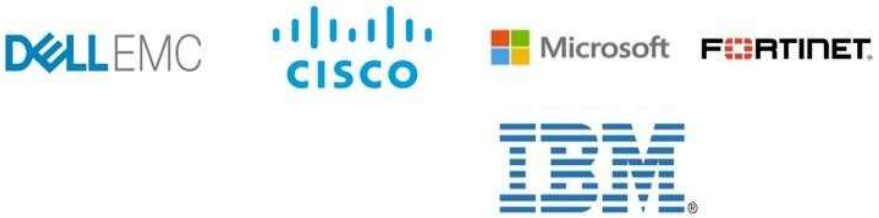


Figure 3 : Logos des partenaires de 3S

3. Étude de l'existant :

3.1. Analyse de l'existant (outils actuels) :

Avant l'approche DevOps, il n'y avait presque pas de communication entre l'équipe de développement et l'équipe système dans le cycle de vie du produit. En effet, l'équipe de développement logicielle ou applicative est responsable de la collecte des besoins métiers et de les développer. Ensuite, une fois le logiciel ou l'application terminé, il sera testé. En contrepartie, si le logiciel ou l'application convient aux besoins métiers, le code source est déposé à l'équipe système pour la tâche exploitation.

Cela crée un problème, parce que sans la communication continue, durant le cycle de vie, des deux équipes, personne ne peut être averti des obstacles que chacun pouvait avoir. Le domaine de l'informatique se développe rapidement et il faut que ce logiciel ou application soit mis en production et sur le marché le plus rapidement possible.

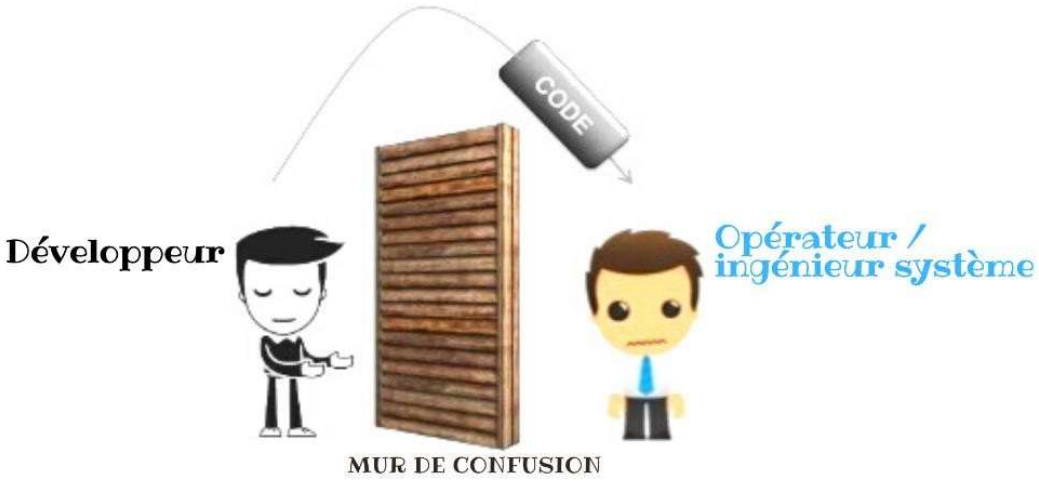


Figure 4 : Avant DevOps

Chaque mise à jour ou une nouvelle fonction doit être déployée sans aucun problème et s'il y a des erreurs, il faut savoir les corriger. L'équipe Ops doit également savoir s'adapter avec un minimum de temps au suivi et au déploiement de ce produit. Ce qui explique l'utilité de l'association du « Dev » et « Ops » ! [2]

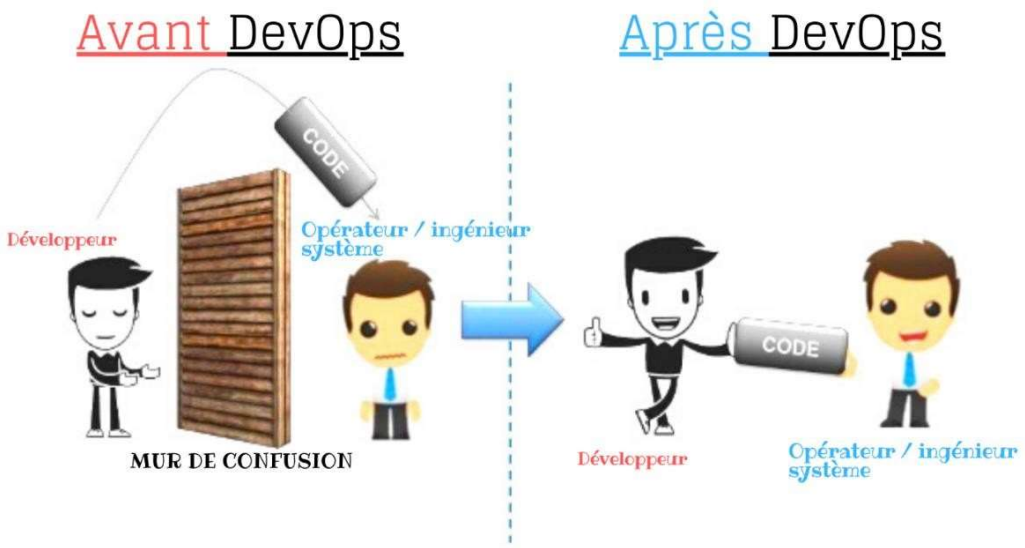


Figure 5 : Association Dev et Ops

3.2. Limites de l'existant :

Après avoir analysé l'existant, nous pouvons tirer ses limites :

- Absence de communication entre l'équipe de Dev et l'équipe Ops durant le cycle de vie du produit. En effet, le code source n'est mis à la disposition de l'équipe Ops que si le logiciel ou l'application répond aux besoins métiers.
- Les tests unitaires ne sont faits que lorsque l'application est finalisée, ceci revient à l'absence d'un serveur de mesure de la qualité de code au cours de la phase de développement.
- Absence d'un outil d'automatisation de déploiement dans les différentes instances / conteneurs.

Avec toutes ces lacunes, ceux-ci nous poussent à changer la méthodologie de travail et s'adapter vis-à-vis au marché.

3.3. Solution proposée :

L'objectif de ce projet est d'introduire la culture d'intégration continue et déploiement continu ou ce qu'on appelle aujourd'hui DevOps et essayer de casser cette barrière du fonctionnement en silo. En effet, une démarche DevOps vous fera gagner du temps voire beaucoup de temps. C'est dans ce contexte que 3S nous a confié la tâche de faire la conception et la mise en place d'une plateforme d'intégration continue et déploiement continu CI/CD pour les applications web multi-services.

Cette solution s'articule autour de 2 parties :

- La première partie c'est la partie d'intégration continue, où il y aura la mise en place d'un serveur d'orchestration Jenkins, un serveur gestion de dépôt Nexus et un serveur de contrôle qualité du code Sonar.
- La deuxième partie est pour la partie de déploiement et la livraison continue. Il s'agit de mettre en place un serveur Ansible, un serveur Docker d'une part et d'autres part 3 serveurs Kubernetes (1 master et 2 nœuds).
- La solution ne s'arrête pas là, il y aura par la suite des scripts d'automatisation développés sous forme de « Playbooks » sur Ansible.
- Nous finissons par une interface web qui aura la responsabilité de gérer l'accès à ce pipeline sans avoir besoin d'une connaissance préalable dans le cloud pour l'équipe de développement.

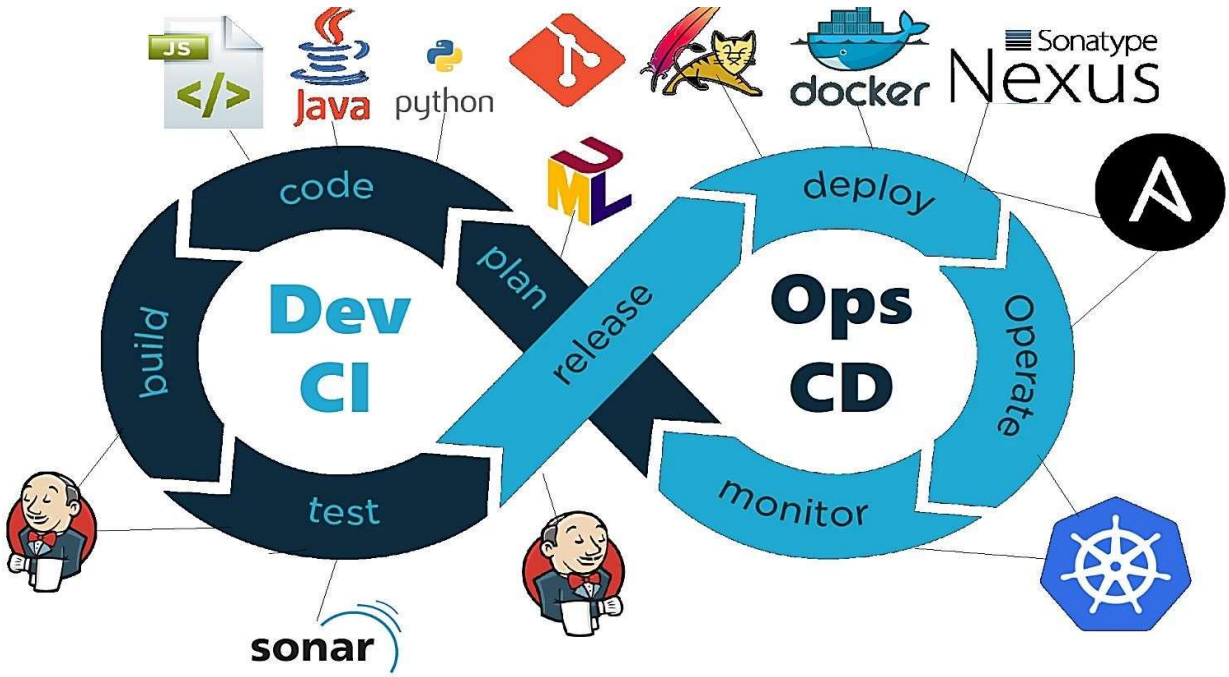


Figure 6 : Solution DevOps

4. Architecture générale de la solution :

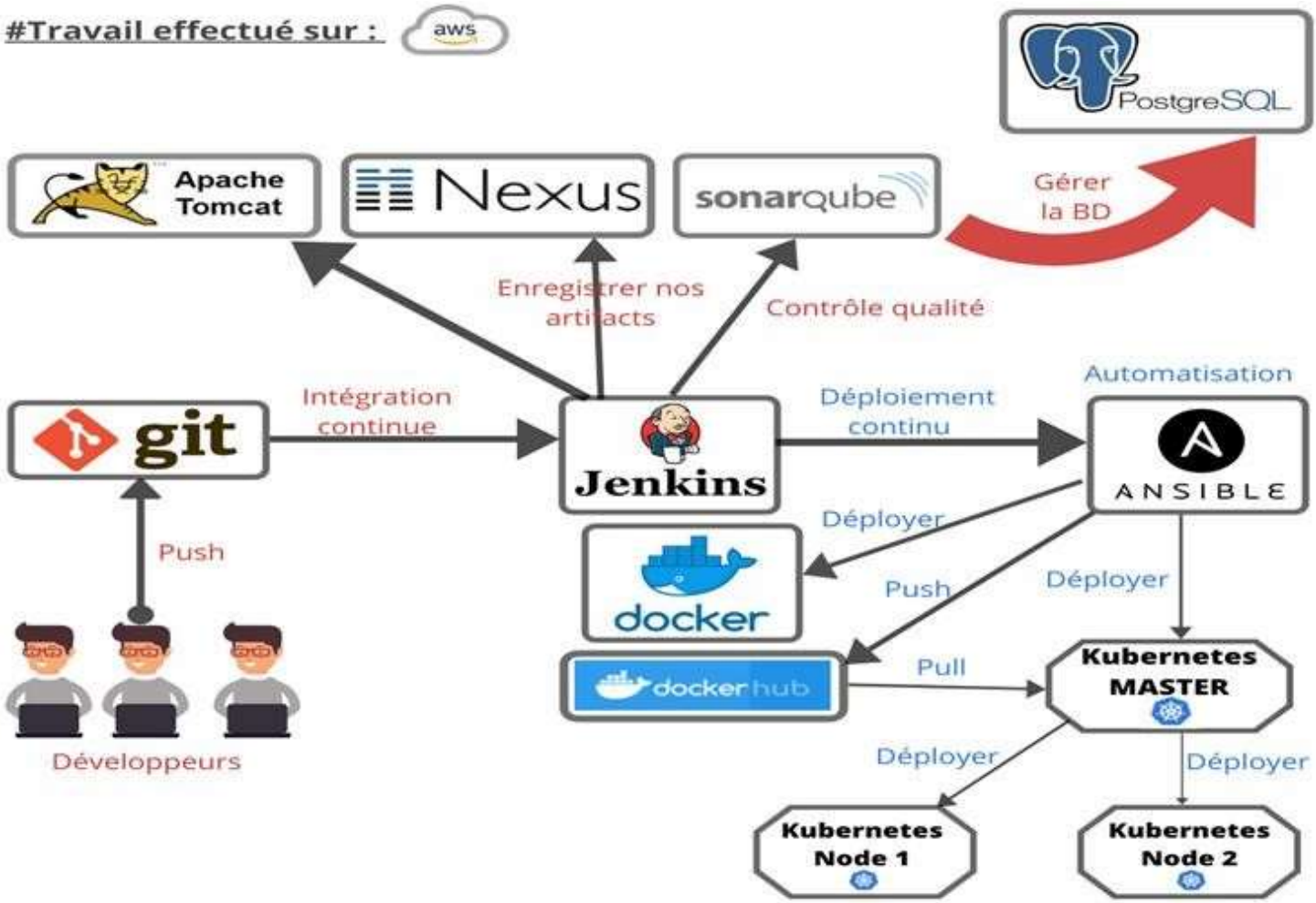


Figure 7 : Architecture générale de la solution

4.1. Git :

Git est un système de contrôle de version, open source, distribué pour suivre les modifications du code source dans n'importe quel ensemble de fichiers pendant le développement du logiciel. Il a pour objectif de coordonner le travail entre les programmeurs. Exemple d'hébergeur référentiel Git : GitHub, GitLab, BitBucket, … [3]



Figure 8 : Logos de Git, Github et GitLab

4.2. Jenkins :

C'est avec ce serveur que nous allons gérer tous les composants (Serveurs, Git, …) de l'intégration continue mais aussi du déploiement et livraison continue. Il est considéré le moteur avec qui se repose ce projet. D'ailleurs, entre plusieurs solutions, nous avons choisi Jenkins car il est le plus utilisé et le leader du marché (Voir chapitre 2. Section étude comparative). [4]

Ce serveur a plusieurs tâches à effectuer :

- ✓ Chercher le code de l'application depuis notre répertoire Git.
- ✓ Effectuer un « build » et des tests unitaires.
- ✓ Envoyer une copie de ce code vers le serveur Nexus.
- ✓ Lancer une tâche pour exécuter le contrôle qualité avec SonarQube.
- ✓ Le cycle de vie du projet ne s'arrête pas là, il notifie le serveur Ansible pour que ce dernier s'occupe du déploiement et la configuration automatisée.
- ✓ Il y aura d'autres sous étapes, tel que l'envoie une copie du code sous forme d'une image Docker vers DockerHub, déploiement vers Tomcat, …



Figure 9 : Logo de Jenkins

4.3. Sonar :

Pour la partie contrôle qualité du code, nous avons choisi comme serveur : SonarQube. En effet, Sonar peut générer plusieurs détails ainsi qu'il nous suggère des alternatives sur le code sous forme d'un rapport présenté dans un tableau de bord, en spécifiant : [5]

- Types des fichiers dans le répertoire
- Nombre des lignes
- Duplications
- Couverture du code en pourcentage
- …

Avec tout ça, Sonar donne un statut global sur le résultat final de l'analyse : « PASSED » (Si le pourcentage est supérieur à un seuil donné) ou « FAILURE » (Si le pourcentage global est inférieur à un seuil donné). Sonar n'est lancé qu'avec un ordre préalable de la part de Jenkins. De plus, Sonar doit être lié avec un serveur de base de données. **Remarque** : Les dernières versions ne supportent plus MySQL en tant que système de gestion de base de données relationnelle. Pour cela, nous avons utilisé PostgreSQL.



Figure 10 : Logo de SonarQube

4.4. Nexus :

Au lieu de stocker nos artéfacts dans notre serveur Jenkins (qui est destiné à lancer des tâches seulement), nous pouvons rajouter un serveur Nexus qui sert à les conserver, et qui sont par la suite exploités dans la phase de déploiement.



Figure 11 : Logo de Nexus Sonatype

4.5. Ansible :

Le processus d'automatisation est fait sur la base d'un serveur Ansible. En effet, Ansible est exploité pour configurer et déployer les mises à jour sur les postes de travail et les serveurs, le provisionnement cloud, le monitoring intra-service et en gros presque tout ce qu'un administrateur système fait sur une base quotidienne.

Pour mieux comprendre, supposons que nous avons 100 serveurs et nous avons besoin d'installer, à titre d'exemple, les packages de vim et wget dans tous les serveurs.

Premièrement, nous allons suivre ces étapes sur un des serveurs :

- S'authentifier dans le serveur.
- Vérifier si ces 2 packages sont installés ou pas.

- Sinon, l'installer en utilisant ces 2 commandes :
  yum install wget -y
  yum install vim -y

Si nous supposons que cette procédure dure 6 min. => Alors pour 100 serveurs, il s'agit de 600 minutes qui est égal à 10 heures.

Supposons aussi, que nous sommes bon au niveau Shell script et que nous pouvons réduire le temps d'exécution en 3 min (écrit et exécution), donc un total de 300 minutes pour les 100 serveurs, qui est égal à 5 heures.

⇨ Ici intervient Ansible, avec cet outil, tout ce processus prend seulement 3 min au lieu de 5 heures ou 10 heures dans le pire des cas.

Ansible va exécuter des tâches parallèlement, ce qu'on appelle « Playbooks » écrits en Python/YAML dans tous les serveurs, et donc nous gagnerons 100 fois le temps. [6]



Figure 12 : Logo d'Ansible

4.6. Docker :

Docker utilise les fonctionnalités du noyau (les espaces de noms et les groupes de contrôle) d'un système d'exploitation (Linux, Windows ou MacOs) pour créer des conteneurs au-dessus d'OS hôte. Ces conteneurs sont isolés les uns des autres et regroupent leurs propres bibliothèques et fichiers de configuration. [7]

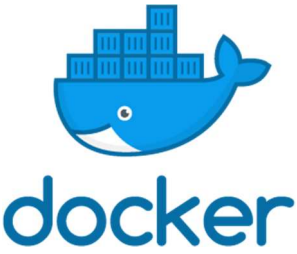


Figure 13 : Logo de Docker

4.7. Kubernetes :

Kubernetes est un orchestrateur de conteneurs open source qui a été développé par Google pour gérer les applications qui s'exécutent dans un environnement de conteneur.

En tant qu'orchestrateur de conteneurs, nous pouvons citer ses tâches :

- Fédérer plusieurs hôtes de calcul, ou nœuds, en une seule cible, appelée cluster,
- Répartir les conteneurs sur les nœuds,
- Répartir les demandes des clients sur plusieurs conteneurs,
- Découvrir les services,
- Répliquer les nœuds et les conteneurs pour gérer une charge de travail accrue, et
- Remplacer les nœuds et conteneurs malsains.

Kubernetes automatise le déploiement, la mise à l'échelle et le fonctionnement des conteneurs d'applications et fournit des outils et des API pour gérer les charges de travail de production sur les conteneurs. [8]



Figure 14 : Logo de Kubernetes

5. Conclusion :

Durant ce chapitre, nous avons commencé par le cadre général du projet en présentant l'organisme d'accueil Standard Sharing Software (3S), ensuite nous avons étudié la situation avant l'apparition de l'approche DevOps et nous avons par la suite tiré les limites de l'existant. Enfin, nous avons expliqué en bref la solution proposée en présentant son architecture et chaque outil utilisé.

Le chapitre suivant mettra plus de lumière sur la démarche DevOps adoptée et les concepts de l'intégration continue et le déploiement continu qu'on va les utilisés dans notre pipeline.

CHAPITRE 2 : ETAT DE L'ART ET CONCEPT DE DEVOPS

CHAPITRE 2 : ETAT DE L'ART ET CONCEPT DE DEVOPS

1. Introduction :

Dans ce chapitre, nous mettons en évidence les différentes notions de base à savoir pour la réalisation de notre projet. D'ailleurs, nous évoquons les concepts de l'intégration continue, le déploiement continu, le cloud computing et la virtualisation.

En plus, nous avons besoin d'un ensemble d'outils, mais face à l'élargissement du choix dans le marché, choisir la combinaison gagnante reste une tâche laborieuse. Pour cela, nous terminons par une étude comparative pour choisir les meilleurs outils qui conviennent aux besoins techniques de notre projet.

2. Intégration continue :

2.1. Définition :

L'intégration continue est un ensemble de pratiques qui a pour objectif de vérifier que chaque modification de code source ne produit pas d'erreurs dans l'application développée. Ce sont des pratiques utilisées en génie logiciel. Dans une entreprise, il y a généralement plusieurs développeurs travaillant sur le même code, et même sur la même application. Avant que Git ne devienne le contrôle de code source distribué, il y avait d'autres outils, tels que SVN ou CVS, mais le référentiel était centralisé, ce qui a entraîné de nombreux problèmes avec l'intégration du code. L'objectif de l'intégration continue est de découvrir ces problèmes d'intégration au plus tôt du cycle de développement.[9]

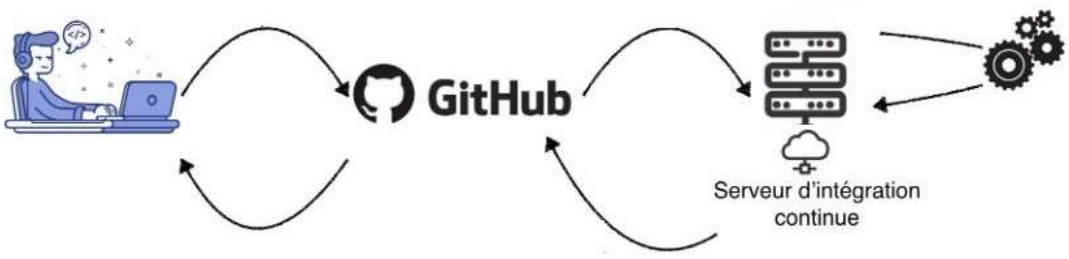


Figure 15 : Concept général de la CI

2.2. Étapes de l'intégration continue (CI)

Afin de détecter ces problèmes, il est nécessaire de passer par plusieurs étapes que nous allons les détailler au fur et à mesure.

En effet, l'intégration continue va se faire en 5 étapes :

1) Commettre le code source dans le répertoire.
2) Récupération du code par le serveur d'intégration continue.
3) Compilation et exécution des Tests de votre code.
4) Mesurez la qualité de votre code et générer un tableau de bord (Rapports).
5) Création de l'artéfact. [10]



Figure 16 : Architecture générale de la CI

2.2.1. Étape 1 : Commettre le code source



Figure 17 : Étape 1 de la CI

La figure 17 montre que le développeur code sa fonctionnalité ou le module qui lui a été défini. Après avoir conçu son module, il effectue des tests unitaires sur sa machine pour s'assurer que tout fonctionne normalement dans son environnement. Il cherche le code sur le gestionnaire de source pour mettre à jour le code actuel. Il fusionne les deux codes (l'ancien et le récupéré) puis commence à résoudre tous les conflits, teste de nouveau son code sur sa machine et enfin apporte des corrections. Si tout va bien, le développeur va procéder à son commit.

2.2.2. Étape 2 : Récupération du code

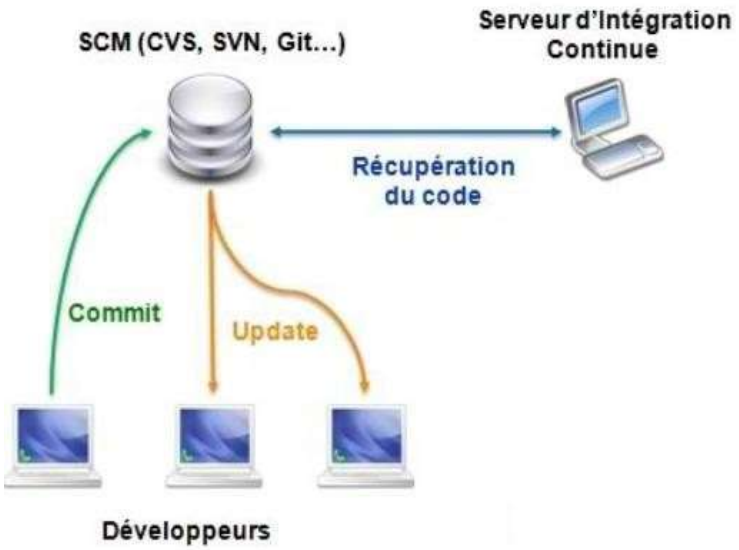


Figure 18 : Étape 2 de la CI

D'après la figure 18, le serveur d'intégration a un service de détection de modification de code. Suite à la dernière publication du développeur dans son répertoire, ce serveur prépare une tâche qui consiste à récupérer la dernière mise à jour du code développé et l'intègre dans sa plateforme. Il lance cette tâche appelée souvent « Job ».

2.2.3. Étape 3 : Compilation/Exécution des tests

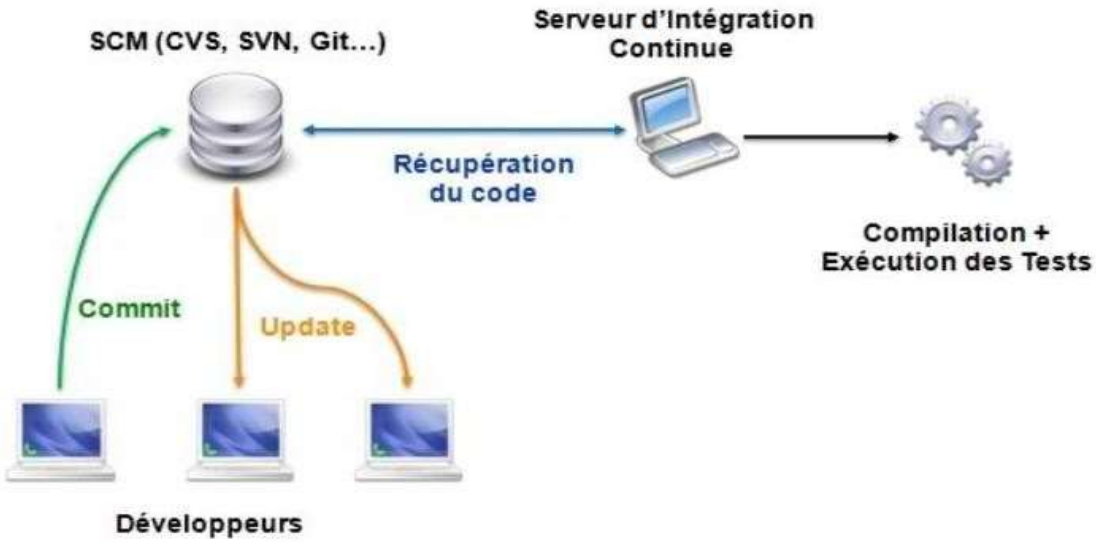


Figure 19 : Étape 3 de la CI

La figure 19 montre qu'une fois le « job » exécuté, la compilation et des tests unitaires seront par la suite déclenchés, la stabilité ou encore les divers bugs pouvant être rencontrés, sont générés et affichés aux équipes Ops et Dev.

2.2.4. Étape 4 : Mesure de qualité

Une fois la tâche terminée, des rapports portant sur la qualité, la stabilité ou encore les divers bugs sont générés dans un tableau de bord, à l'aide d'un serveur de contrôle de qualité qui sera lié à ce serveur d'intégration continue, et qu'avec ce tableau de bord que l'équipe va voir proprement l'état général du code et surtout prendre des décisions pour l'amélioration de la performance et la qualité du code.

En effet, L'équipe peut alors consulter ces rapports, les analyser, traiter des bugs s'il y en a, puis continuer à développer les autres phases du projet.

En réalité, notre serveur d'intégration continue peut nous avertir sur la qualité en faisant ses tests unitaires mais de façon limitée. C'est pourquoi, que nous devons rajouter un serveur de mesure de qualité qui mieux décrit ces données importantes en un tableau de bord simple, clair et organisé, comme le montre la figure ci-dessous :



Figure 20 : Étape 4 de la CI

2.2.5. Étape 5 : Création de l'artéfact

Dans la figure 21, le serveur d'intégration continue passe à créer des artéfacts du code source et les stockent dans un serveur de gestion des dépôts binaires. Cette tâche que nous allons effectuer, permettra de récupérer le package de l'application rapidement, et éviter la tâche de retélécharger tout le code depuis notre répertoire, et de ne retélécharger que les modifications détectées par le serveur d'intégration continue.

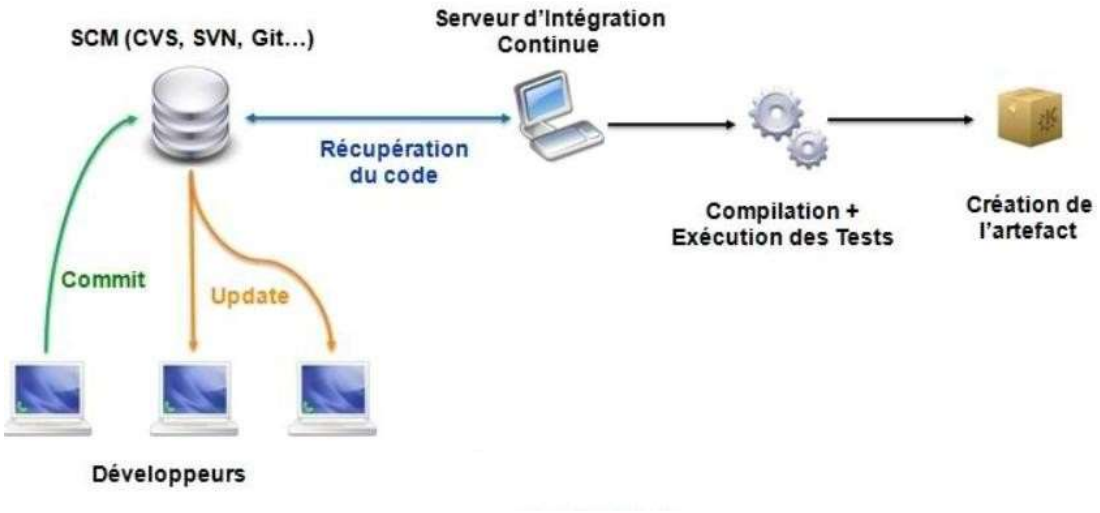


Figure 21 : Étape 5 de la CI

2.3. Composants de la CI :

Comme nous avons remarqué dans les étapes de la CI, il y a des exécuteurs de ce mécanisme, nommés comme le montre la figure 22 : Ordonnanceur, VCS, Compilateur, …

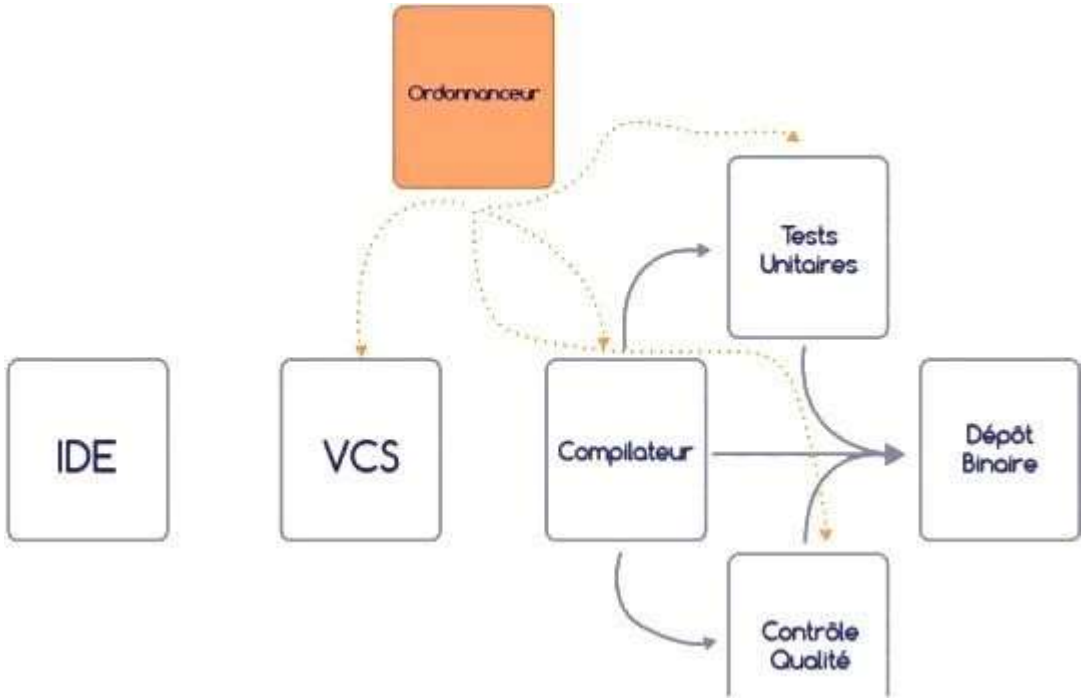


Figure 22 : Composants de la CI

- **L'ordonnanceur** : Ou en d'autres termes le serveur moteur de la CI, c'est un chef d'orchestre. Il ordonne les tâches pour qu'elles se réalisent dans le bon ordre et surtout il respecte le tri mis par l'administrateur. Il déclare également les erreurs aux utilisateurs après les tests. Exemple : Jenkins, Bamboo, …

- **Le gestionnaire de source (VCS)** : C'est le lieu où se retrouve l'ensemble des codes sources sous différentes versions, sous forme des répertoires bien organisés suivant le choix des utilisateurs. Tous les développeurs doivent commettre leurs codes ici. Exemple : Git (GitHub, GitLab), SVN, …
- **Le compilateur** : C'est un outil qui nous permet de faire la traduction du code (par exemple du C ou Java, …) en langage compréhensible par la machine, ou par un interpréteur.
  Exemple : GCC, JDK, ...
- **Les tests unitaires** : Ce composant exécute un ensemble de tests unitaires définis dans un projet.
- **Le contrôle qualité** : Ce composant est responsable de la qualité du code en fonction de plusieurs paramètres qui l'exposent en un tableau de bord ou en des alertes, tels que : Nombre d'erreurs, Bugs, … Exemple : SonarQube, …
- **Le dépôt de binaire** : Ce composant stocke les différentes versions d'un projet après compilation afin de les mettre en communs avec tous les utilisateurs. Il servit aussi comme source pour des composants utilisés dans son propre projet. Exemple : Nexus, …

2.4. Avantages de la CI :

Avec cette technique de CI, nous gagnerons en productivité, en temps, en argent, mais aussi en qualité de code.

- **Productivité :** En plus de l'identification rapide des bugs avant la mise en production du logiciel, la CI permet de livrer plus rapidement des versions et des mises à jour.
- **Temps :** Au lieu de tester en dernier lieu notre code, les développeurs gagnent beaucoup de temps à repérés et corrigés plus rapidement entre temps, grâce à l'utilisation des tests unitaires.
- **Argent :** En plus d'un pipeline automatisé, donc un gain de temps et d'argent, la CI offre aussi d'autres fonctionnalités qu'avec eux nous pouvons réduire l'équipe.
- **Qualité de code :** Comme son nom l'indique, la qualité de code entre en jeu dans un pipeline CI.

3. Déploiement continu /Livraison continue (CD)

3.1. Définition :

La livraison continue (respectivement le déploiement continu) est une extension de l'intégration continue. En effet, le CD se concentre sur l'automatisation du processus de livraison des applications afin que les équipes puissent au plus tôt déployer et en toute confiance leur code en production facilement. En veillant à ce que la base de code soit toujours dans un état déployable, publier une application se transforme en un événement banal sans rituel compliqué. Les équipes peuvent chaque fois qu'elles en ont besoin à livrer, sans coordination complexe ni test en phase avancée. [11]

3.2. Déploiement continu Vs Livraison continue

La différence entre la livraison continue et le déploiement continu consiste dans la présence d'un processus manuelle pour mettre à jour et produire. Avec le déploiement continu, la production se fait automatiquement, sans une intervention explicite.

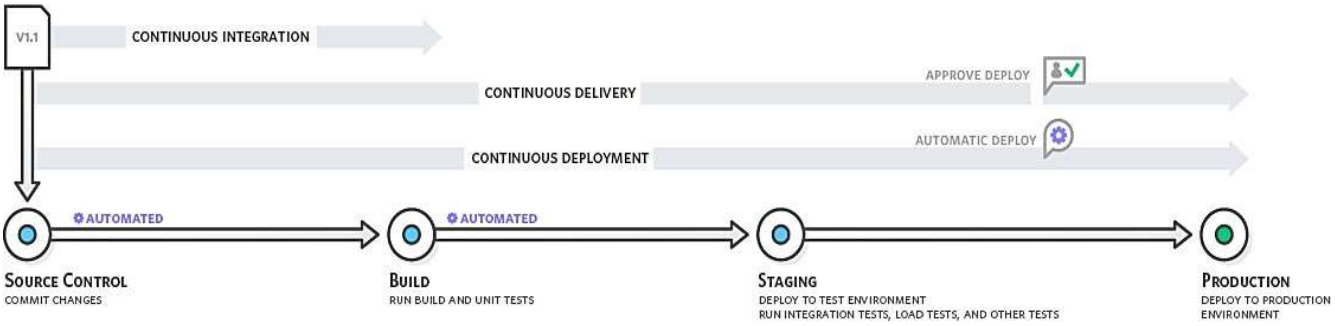


Figure 23 : Différence entre CD et cd

3.3. Étapes du CD :

Le déploiement continue et/ou la livraison continue se font en 6 étapes comme le montre la figure ci-dessous, qui décrit un exemple réel d'un CD avec les outils les plus utilisés en ce moment.

Les 6 étapes sont :

1) Le développeur va commettre son code dans son répertoire.
2) Le serveur CI remarque un changement et lance un build.
3) Le serveur CI va tirer le code depuis le répertoire.
4) Le serveur CI avec l'aide du serveur d'automatisation va s'occuper du push vers DockerHub.
5) Le serveur d'automatisation, vérifie le succés du push et lance un job successif.

6) Le serveur d'automatisation, déploie finalement l'application dans les serveurs.

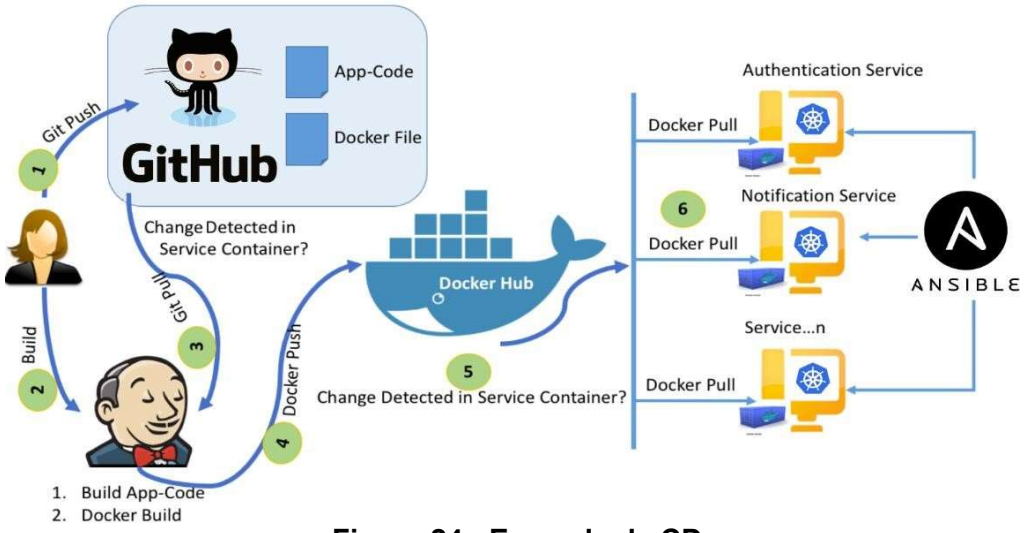


Figure 24 : Exemple de CD

3.3.1. Commettre son code :

Durant cette étape, le développeur commet son code dans un répertoire de gestion des codes sources. Exemple : GitHub, GitLab, SVN, Bucket, …

3.3.2. Détection d'un changement :

Une fois le code est commis dans le répertoire, le serveur CI remarque un changement et essaye de détecter et d'apporter le changement dans le code. Pour cela, il lance un build.

3.3.3. Tirer son code :

Après, le serveur CI tire son code depuis ce répertoire.

3.3.4. Push vers DockerHub :

Le serveur CI, lance un autre build, en donnant l'ordre à un autre serveur d'automatisation (Exemple : Ansible, Chef, …) pour faire le push vers un répertoire Docker qui est souvent dans DockerHub.

3.3.5. Serveur d'automatisation continue le déploiement :

Après la phase du « push », un autre job sera lancé par la suite, pour terminer le travail du déploiement. Mais ce job ne sera lancé qu'en cas de sucés du job précédant du push.

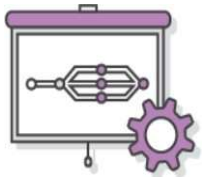
3.3.6. Déploiement vers les serveurs cibles :

L'application ou le code sera proprement déployé dans les serveurs cibles qui sont forcément orchestrés par un outil tel que Kubernetes ou bien manuellement, en utilisant la virtualisation en conteneurs avec Docker.

3.4. Avantages du CD :

Nous pouvons tirer plusieurs avantages du CD, nous notons par exemple : [12]

- **Automatisation du processus de déploiement logiciel :** Le déploiement continu nous permet de se procéder à une configuration automatique à travers la création, le test et les changements de code en vue d'une mise en production, afin d'améliorer l'efficacité et la rapidité de nos projets de livraison de logiciel.



- **Amélioration de la productivité :** Ces pratiques aident l'équipe de développement à gagner en productivité, en limitant le nombre de tâches manuelles qui doivent être accomplies par eux-mêmes et aussi en encourageant les habitudes et actes qui nous aident à réduire le nombre d'erreurs.



- **Trouver et corriger plus rapidement les erreurs** : Les membres de l'équipe peuvent découvrir et corriger rapidement les erreurs, grâce à des tests plus fréquents et plus poussés, avant qu'ils ne prennent de dimension. Le CD nous permet de mener facilement d'autres types, car tout le processus a été automatisé.



- **Livrer plus rapidement des mises à jour :** La livraison continue aide notre équipe à livrer des mises à jour aux clients plus rapidement et plus fréquemment. Une bonne livraison continue nous permet de toujours suivre un processus de test standardisé, prêt à déployer des artefacts à tout moment.



4. Cloud computing – AWS :

4.1. Définition :

Le cloud computing met à la disposition des utilisateurs, des ressources informatiques à la demande via Internet en un ensemble de services (telles que la puissance de calcul, le stockage et les bases de données) avec une tarification basée sur notre utilisation de chacun de ces services. Nous pouvons accéder à notre guise aux services technologiques au lieu d'acheter, posséder et de gérer des serveurs et des centres de données physiques.



4.2. AWS & Services utilisés dans le projet :

AWS ou bien Amazon Web Services est une filiale d'Amazon qui offre à la demande, sous forme de services, des plates-formes et des API d'informatique.



Figure 25 : Logo AWS

Nous allons définir les services utilisés dans ce projet, en particulier :

4.2.1. Service EC2 :

EC2 ou Amazon Elastic Compute, c'est le service primordial de ce projet, et d'ailleurs il constitue la partie centrale de tout le cloud d'Amazon. C'est avec ce service que nous pouvons allouer des différentes instances (ordinateurs virtuels) à la demande et avec les caractéristiques que nous voudrons sur lesquelles nous exécutons et nous contrôlons nos propres applications informatiques.

4.2.2. AMI :

AMI ou Amazon Machine Image, c'est un sous service dans EC2, pour déplacer nos instances, en donnant l'accès aux utilisateurs de notre choix, d'une VPC (Virtuel Private Cloud) à un autre et d'un compte à un autre sans être obliger de réinstaller et configurer tout le travail.

4.2.3. Adresses IP Élastiques :

Quand nous arrêtons nos instances et nous les redémarrons, nous remarquons toujours qu'il y aura un changement d'adresses IP publique, ce qui perturbe la configuration. Avec les IP élastiques nous pourrons fixer nos adresses.

4.2.4. Service RDS :

RDS ou Amazon Relational Database Service, est un service de base de données relationnelle fait pour simplifier la mise en place, la configuration et le fonctionnement d'une base de données relationnelle à utiliser dans les applications.



Figure 25-b : Services AWS

5. Virtualisation :

5.1. Définition :

En informatique, la virtualisation est conçue à faire fonctionner plusieurs systèmes, serveurs ou applications sur un même serveur physique. La virtualisation est un constituant technique important et clé dans le Cloud Computing.

Nous définissons : Machine hôte = machine fonctionnant dans différents systèmes virtuels, machine invitée = machine virtuelle fonctionnant dans l'environnement de virtualisation. [13]

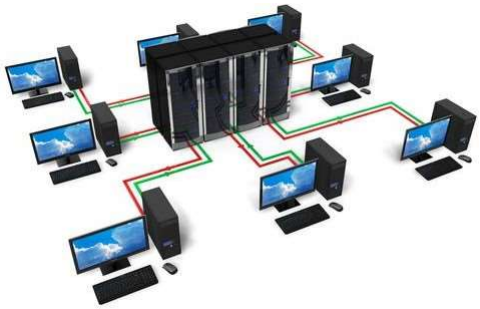


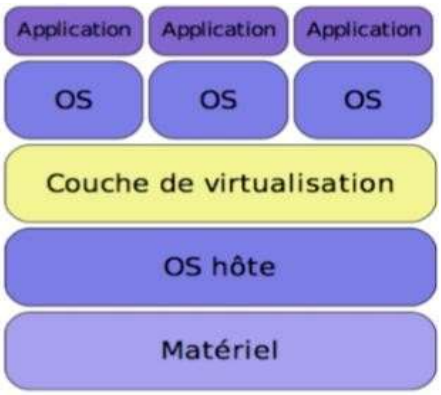
Figure 25-c : Virtualisation

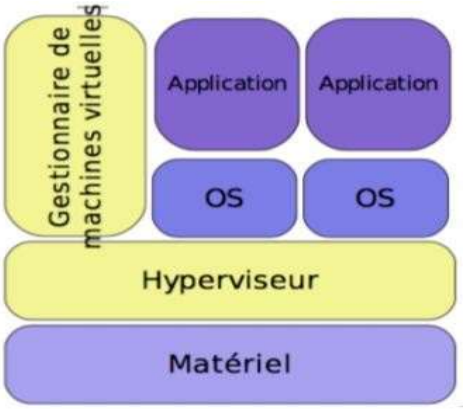
5.2. Domaines de virtualisation :

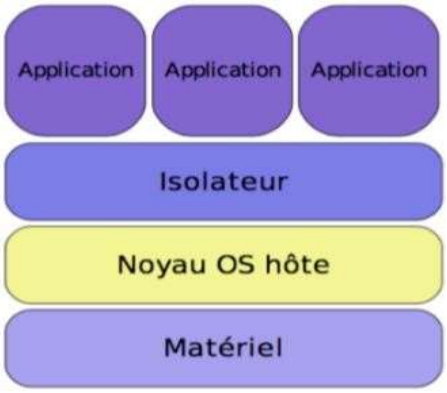
Il existe 4 grands principaux domaines dans la virtualisation :

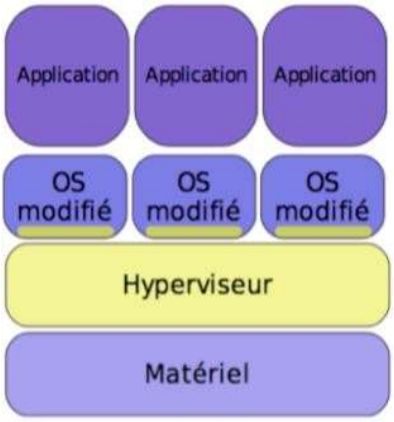
- **Virtualisation d'applications (du contexte d'exécution)** : C'est le fait de séparer l'application du système d'exploitation hôte, des autres applications présentes afin d'éviter les conflits entre eux.
- **Virtualisation de serveur** : Un serveur physique peut être diviser en plusieurs serveurs virtuels. Exemple : Sur un serveur physique nous pouvons créer plusieurs instances Windows Server et dans chaque instance, nous pouvons mettre en place des serveurs de HTTP, DHCP, NTP, DNS, ...
- **Virtualisation du réseau :** Le principe de base c'est de regrouper un ensemble de machines de façon logique et non physique. Exemple : Les **VLAN**s : nous pouvons diviser une adresse réseau en plusieurs, sous forme des VLANs.
- **Virtualisation du stockage** : Le principe de base est de gérer une interface qui permet de dissocier la gestion physique des disques (et des baies de stockage) vis-à-vis des serveurs qui l'utilisent. Exemple : Administration en mode bloc ou en mode fichiers.

5.3. Différentes techniques de virtualisation :

<table>
<tr><th>Technique 1 :
Hyperviseur de type 2</th><th>Technique 2 :
Hyperviseur de type 1</th></tr>
<tr><td>
- Hyperviseur installé sur un OS.
- Indépendance entre les OS invités.
- Performances réduites coté E/S.


Exemples: VirtualBox, QEMU, Vmware (Workstation, fusion, player), Microsoft virtual PC.
</td><td>
- Noyau léger.
- Outils de gestion et de supervision.
- Possibilité d’exécution d’OS natifs.


Exemples : Xen, Vmware Vsphere, KVM, …
</td></tr>
<tr><th>Technique 3 :
Isolation ou Conteneurisation</th><th>Technique 4 :
Paravirtualisation</th></tr>
<tr><td>
- Emprisonner l’exécution des applications dans des contextes d’exécution.
- Solution performante et économique en termes de mémoire.
- Partage du code noyau de l’OS hôte.


Exemples : Docker, OpenVZ, Linux Vserver
</td><td>
Le paravirtualiseur est un noyau hôte optimisé et allégé qui exécute uniquement le noyau invité qui a été ajusté et optimisé.
- Noyaux de la cellule hôte et invité optimisés.
- Peut être utilisé sans instructions spéciales.
- Ne s’applique pas aux systèmes non libres.


Exemple : Hyper-V
</td></tr>
</table>

Tableau 1 : Différentes techniques de virtualisation

5.4. Technique de virtualisation :

Sur la base de la recherche et de la comparaison dans la section qui présente différentes technologies de virtualisation (page 34) : Pour ces raisons, nous avons choisi la technique de **conteneurisation** comme solution pour la livraison et le déploiement continu d'applications :

- Les conteneurs partagent le même système d'exploitation contrairement aux machines virtuelles.
- Les conteneurs sont beaucoup plus portables et efficaces.

Remarque : Nous rappelons que nous avons retenu **Docker** comme moteur de création et exécution des conteneurs et **Kubernetes** comme un outil de monitoring et orchestration de ces conteneurs en vue de simplifier la surveillance et la coordination.

6. Conclusion

Durant ce chapitre, nous avons vu les concepts de base de ce projet. Nous avons présenté la démarche DevOps adopté en expliquant les notions d'intégration continue et de livraison et déploiement continu. Ensuite, nous avons évoqué quelques services utilisés dans AWS, et le concept de la virtualisation comme outil de déploiement utilisé dans ce projet. Après, nous avons fini par une étude comparative en choisissant chaque fois l'outil retenu.

Dans le chapitre suivant nous détaillerons l'étude conceptuelle de ce projet juste avant d'entamer la partie réalisation.

CHAPITRE 3 : ANALYSE ET CONCEPTION UML

CHAPITRE 3 : ANALYSE ET CONCEPTION UML

1. Introduction

Dans ce chapitre, nous abordons l'analyse des besoins et la conception UML qui constitue une étape primordiale qui précède chaque réalisation. Nous entamons par un aperçu sur les objectifs du projet et les besoins fonctionnels et non fonctionnels. Nous procédons ensuite à présenter les acteurs et expliquer chaque cas d'utilisations. Et nous finissons par représenter les diagrammes des composants et de déploiement de ce projet qui expliquent les intéractions entre chaque composant.

1.1. Cas d'utilisation pour le site web :

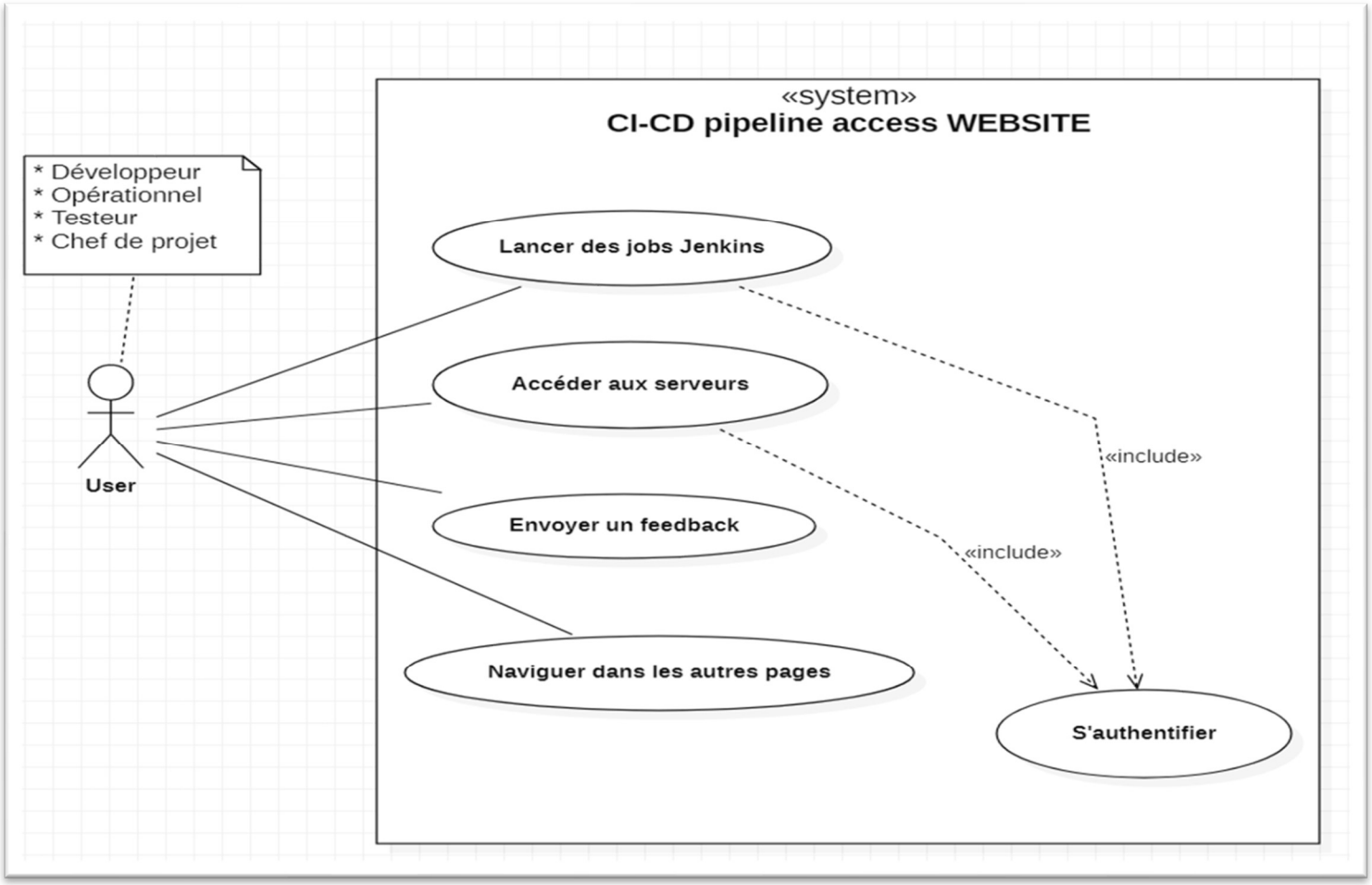


Figure 27 : Diagramme des cas d'utilisations du site web

Cas d'utilisation	Description
Lancer des jobs Jenkins	Une page web dédiée pour lancer des jobs depuis le site sans la nécessité d'accéder au serveur Jenkins.
Accéder aux serveurs	Au lieu de se connecter à AWS, on peut y accéder directement dans le site en une seule clique sur le serveur souhaité.
Envoyer un feedback	En cas d'un problème dans le site, il y aura une page de contact.
Naviguer dans les autres pages	Il existe d'autres page annexes comme la page d'accueil et about.
S'authentifier	Pour lancer des jobs et accéder aux serveurs il faut s'authentifier.

Tableau 9 : Explication des cas d'utilisations (2)

2. UML – Diagramme des composants :

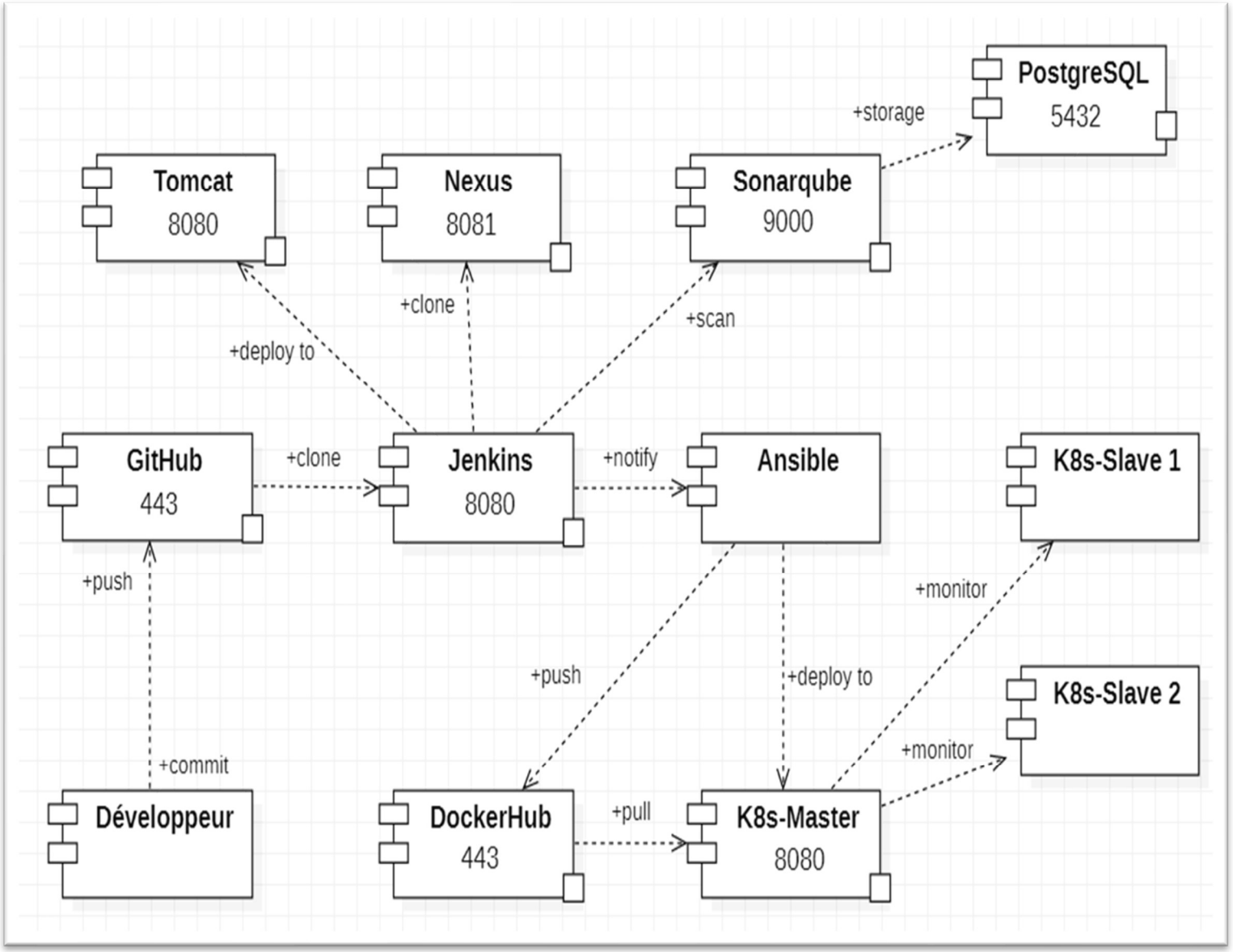


Figure 28 : Diagramme des composants du pipeline

Ce diagramme illustre la séquence des opérations les plus importantes effectuées par notre pipeline. En fait, chaque développeur travaille localement: il promet GitHub pour chaque nouvelle modification du code. Un « build » sera démarré automatiquement sous Jenkins, ce dernier télécharge le code, le compile, puis le scanne à l'aide de Sonarqube, et envoie enfin les livrables générés au serveur Nexus. Dans le même temps, un autre job Jenkins sera lancé à Ansible pour créer une image dans DockerHub et déployer cette image en conteneurs dans les nœuds Kubernetes.

3. UML – Diagramme de déploiement :

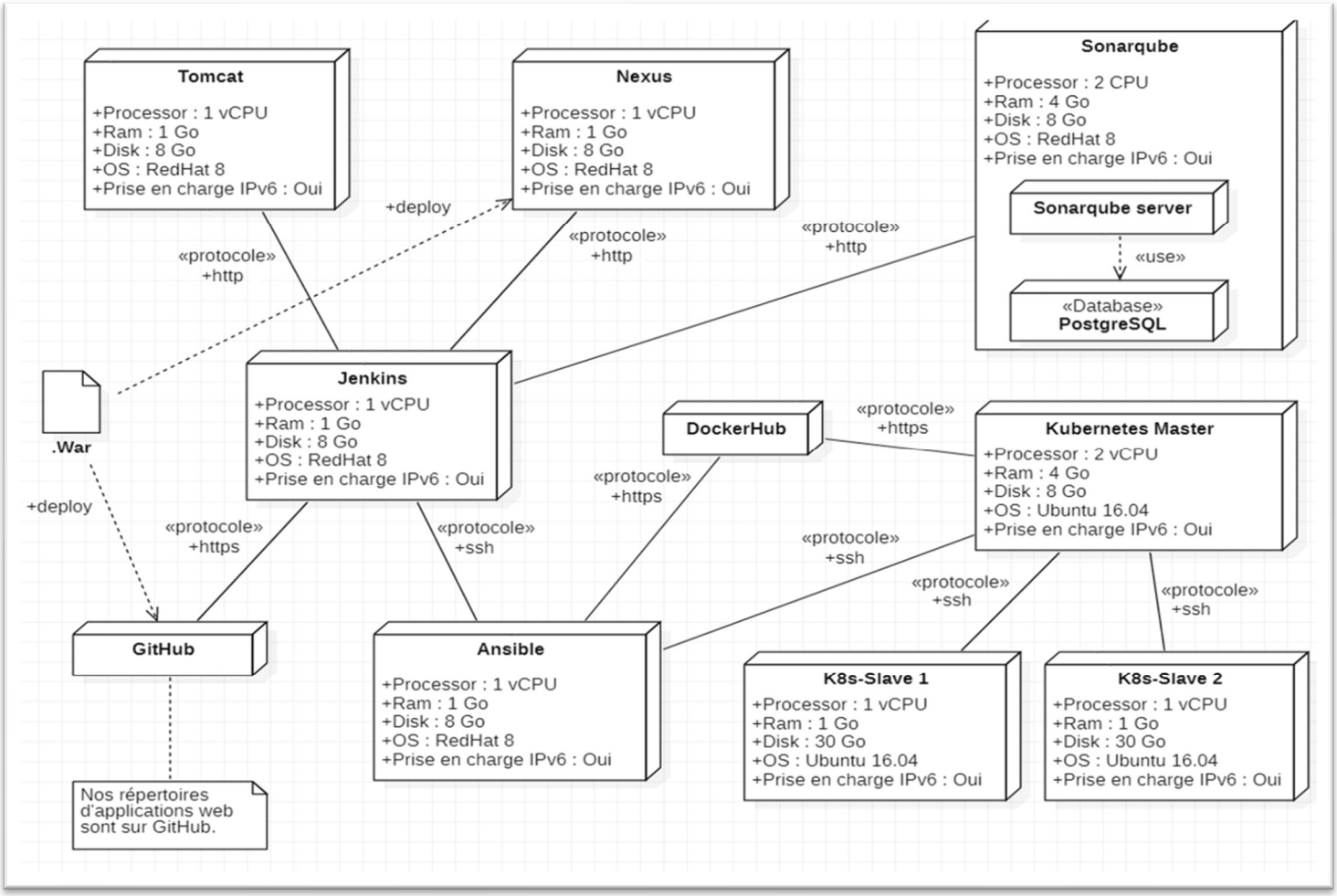


Figure 29 : Diagramme de déploiement du pipeline

Ce diagramme montre les détails de déploiement en mettant en valeur les caractéristiques techniques de chaque instance. En effet, le choix de ces caractéristiques a été fait selon le besoin. Par exemple, un nœud Kubernetes qui va accueillir les applications exige beaucoup plus de stockage que de Ram, ce qui explique notre choix. Ainsi, ce diagramme met en valeur la communication entre les instances et les protocoles utilisés.

4. Conclusion

Ce chapitre nous a permis de détailler l'aspect conceptuel de ce projet avant d'entamer la mise en œuvre. Nous avons spécifié les besoins fonctionnels et non fonctionnels de notre système avant de clôturer avec les diagrammes UML qui illustrent les différents cas d'utilisations et les composants de ce projet, ainsi, le processus de déploiement et différentes les intéractions. Le chapitre qui suit fera l'objet de réalisation de notre projet.

CHAPITRE 4 : REALISATION

CHAPITRE 4 : REALISATION

1. Introduction :

Durant ce chapitre nous entamons la partie réalisation de ce projet, cette phase consiste à transformer le modèle conceptuel établi précédemment en des composants logiciels formant notre système. Nous allons expliquer et présenter nos travaux techniques pour mettre en place cette plateforme.

2. Environnement de travail :

Cloud: Amazon Web Service (AWS).

Services dans le cloud: EC2, RDS, AMI, IP Elastic.

Caractéristiques des serveurs utilisés: **Voir le diagramme de déploiement dans le chapitre 3 figure 29.**

3. Implémentation :

3.1. Mettre en place le 1er élément du pipeline ~ Jenkins :

Après avoir installé Jenkins :

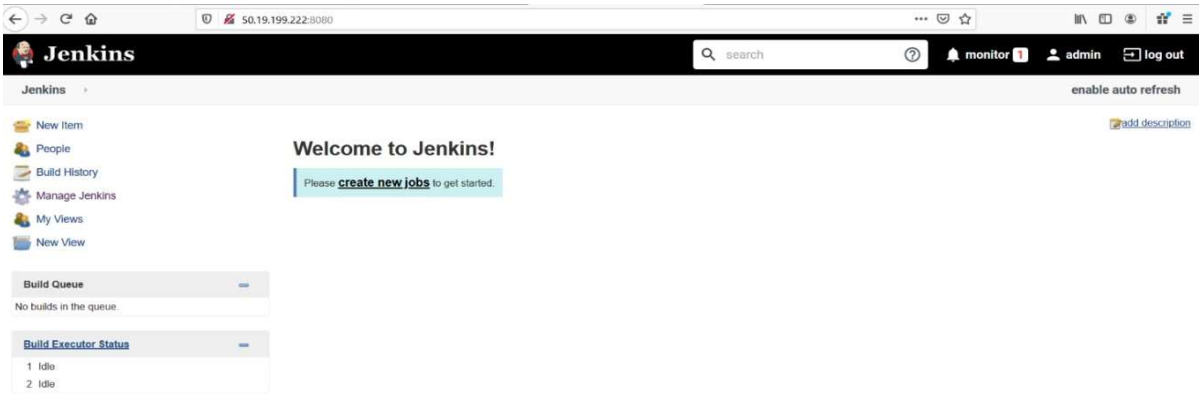


Figure 30 : Page d'accueil Jenkins

Nous lançons un premier JOB de test, pour s'assurer de la bonne installation de Jenkins :

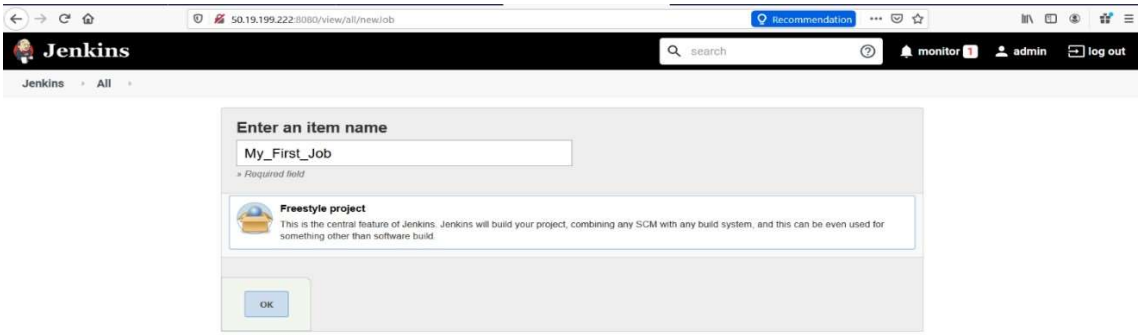


Figure 31 : 1er Job Jenkins

Ce job consiste seulement à faire l'affichage d'un message par notre choix.



Figure 32 : Commande du 1er job

Puis, nous testons le fonctionnement :

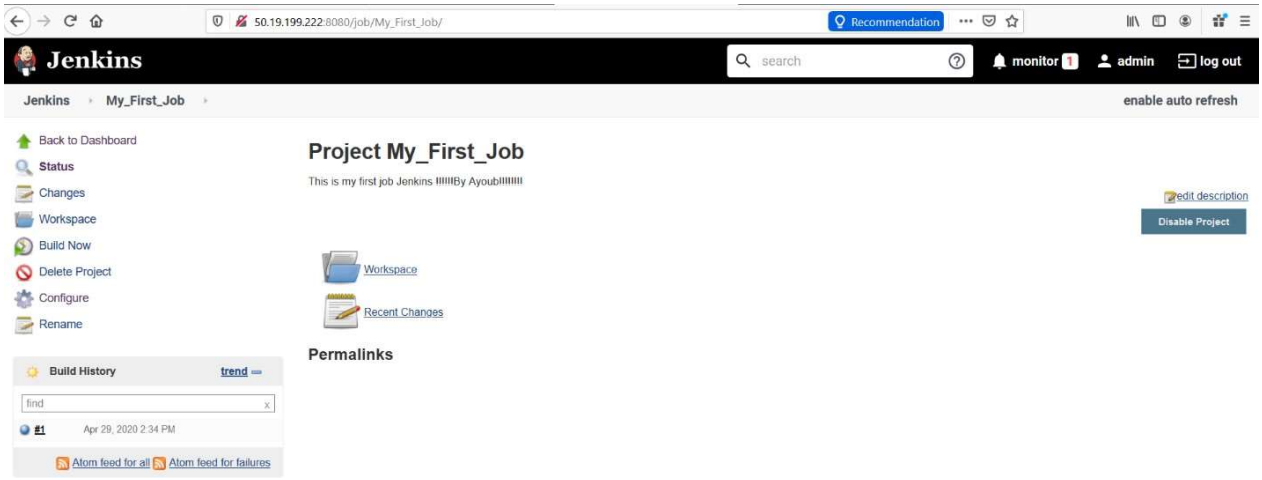


Figure 33 : Test de fonctionnement du 1er job

Nous remarquons la bulle bleue, ce qui explique que la construction de ce test est faite avec succès. Nous devons intégrer Git et Maven avec notre serveur Jenkins. Pour cela, nous allons ajouter les « plugins » nécessaires.

3.2. Intégration d'un serveur Tomcat dans le pipeline :

Après avoir installé Tomcat et après une vérification de sa fonctionnalité, nous configurons ensuite les fichiers context.xml pour pouvoir l'administrer :

```
root@tomcat bin]# find / -name context.xml
/opt/tomcat/webapps/manager/META-INF/context.xml
/opt/tomcat/webapps/host-manager/META-INF/context.xml
/opt/tomcat/conf/context.xml
[root@tomcat bin]#
```

Figure 34 : Configuration context.xml

3.2.1. Déploiement d'un fichier .war dans le serveur Tomcat en créant une tâche Jenkins (Option 1) :

- Après avoir installé aussi le plugin dans Jenkins. Dans cette partie, nous créons une tâche Jenkins intitulé « Deploy_on_Tomcat_Server » qui déploie notre application .war depuis notre répertoire GitHub, vers le serveur Tomcat.

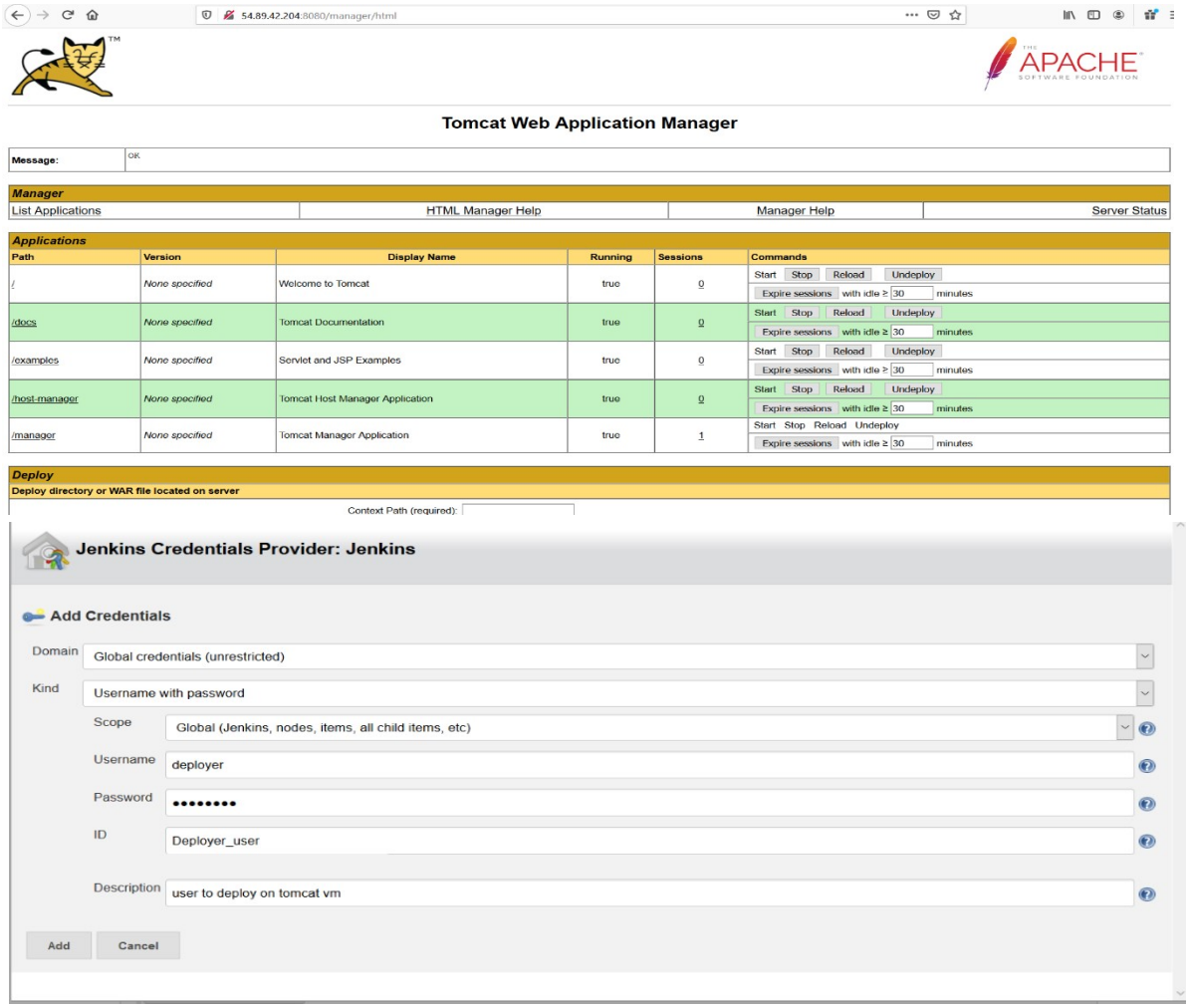


Figure 35 : Authentification avec Jenkins

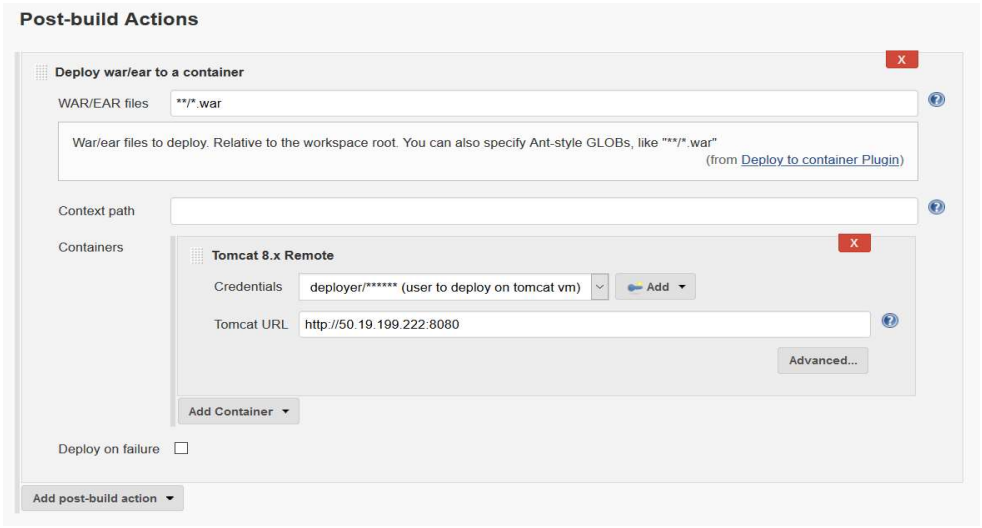


Figure 36 : Action après la construction

Nous lançons le job puis nous vérifions le répertoire dans /opt/tomcat/ dans notre serveur :

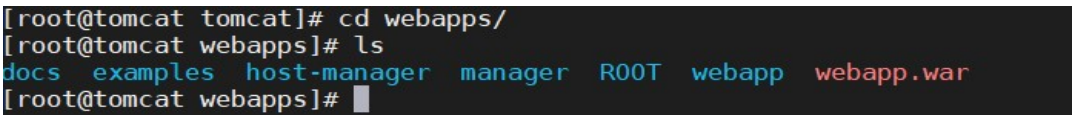


Figure 37 : schéma : /opt/tomcat/

Nous remarquons que le fichier .war est déployé dans le dossier du server.

3.2.2. Automatisation du déploiement (PollSCM) :

Cette technique PollSCM interroge périodiquement le SCM pour vérifier si des modifications ont été apportées (c'est-à-dire de nouvelles validations) et construit le projet si de nouvelles validations ont été poussées depuis la dernière génération. La figure ci-dessous montre que Jenkins doit vérifier chaque minute notre répertoire.

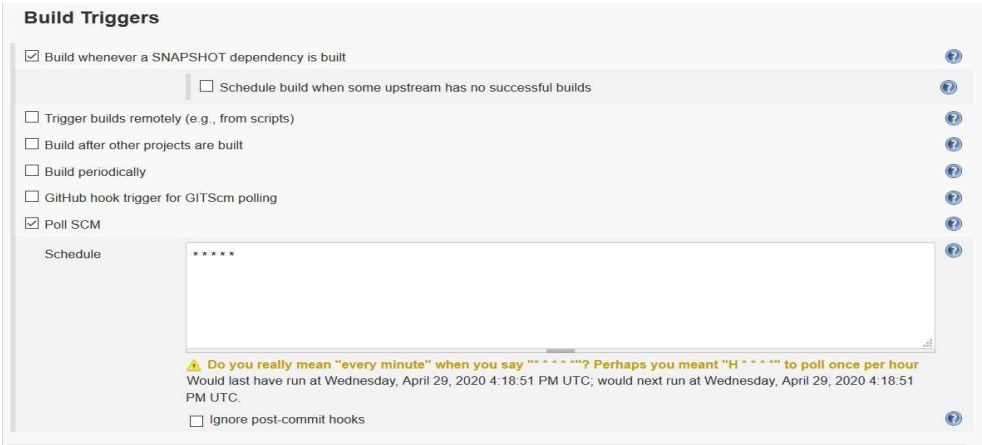


Figure 38 : Configuration du PollSCM

Pour vérifier son fonctionnement, nous devons modifier notre code et nous le poussons vers notre répertoire dans GitHub comme le montre la figure ci-dessous :

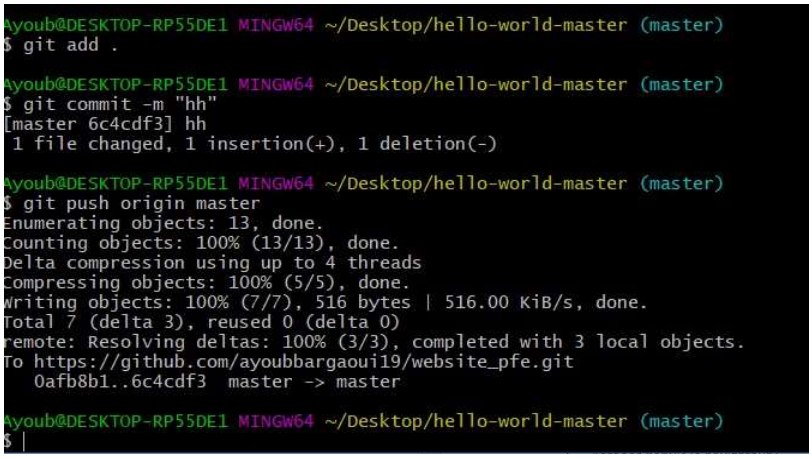


Figure 39 : Push vers GitHub

Et voilà, Jenkins détecte qu'il y a eu un changement dans notre répertoire et un « job » sera lancé automatiquement comme le montre la figure ci-dessous :

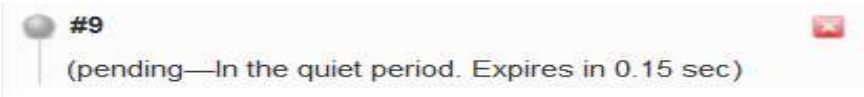


Figure 40 : Job lancé automatiquement

Nous vérifions la création du conteneur en tapant l'adresse ip public dans le navigateur sur le port 8081.

3.3. Intégration de SonarQube dans le pipeline :

Nous lançons une nouvelle instance dans AWS :



Pour la mise en place de ce contrôleur de qualité nous devons suivre ces étapes :

- Nous configurons la base de données dans cette instance et nous l'administrons avec PostgreSQL (Les nouvelles versions de SonarQube ne supportent plus MySQL).



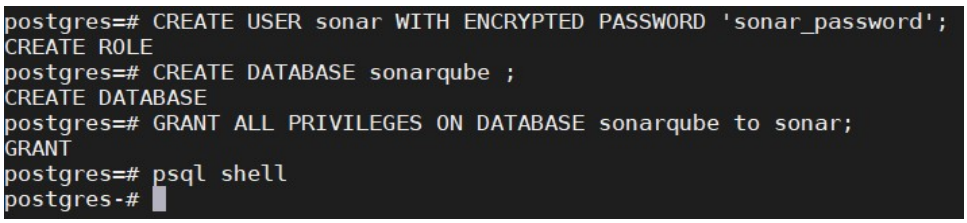


Figure 41-b : Créer un utilisateur et une BD pour Sonar

- La deuxième étape c'est que nous devons installer sonar scanner dans Jenkins.

```
[root@ip-172-31-27-2 opt]# wget https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-4.3.0.2102-linux.zip
--2020-05-03 17:07:54--  https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-4.3.0.2102-linux.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 91.134.125.245
```

- Après avoir installé sonar scanner, nous le configurons depuis Jenkins avec le serveur.

  URL : http://18.208.235.212:9000/sonarqube

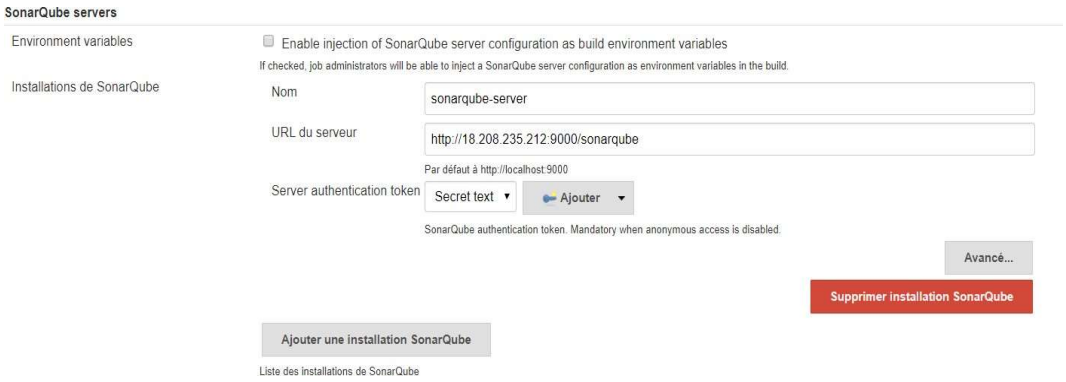


Figure 41-c : Configuration sonar scanner dans Jenkins



Figure 41-d : Clé d'authentification entre Sonar et Jenkins

- Nous créons un « job » et nous le lançons pour exécuter une analyse de contrôle qualité. Nous remarquons que dans un premier temps, le résultat de l'analyse à donner une erreur dans le code.

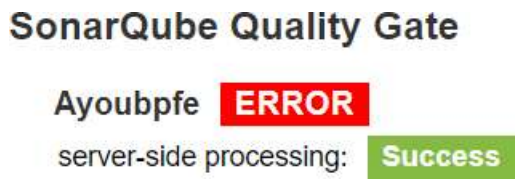


Figure 42 : Résultat de la première analyse sonar coté Jenkins

Nous constatons d'après le tableau de bord, dans la page suivante, que le taux de fiabilité de ce code est inférieur à un seuil prédéfini pour considérer ce code en état « Passed » et non pas « Failed » :

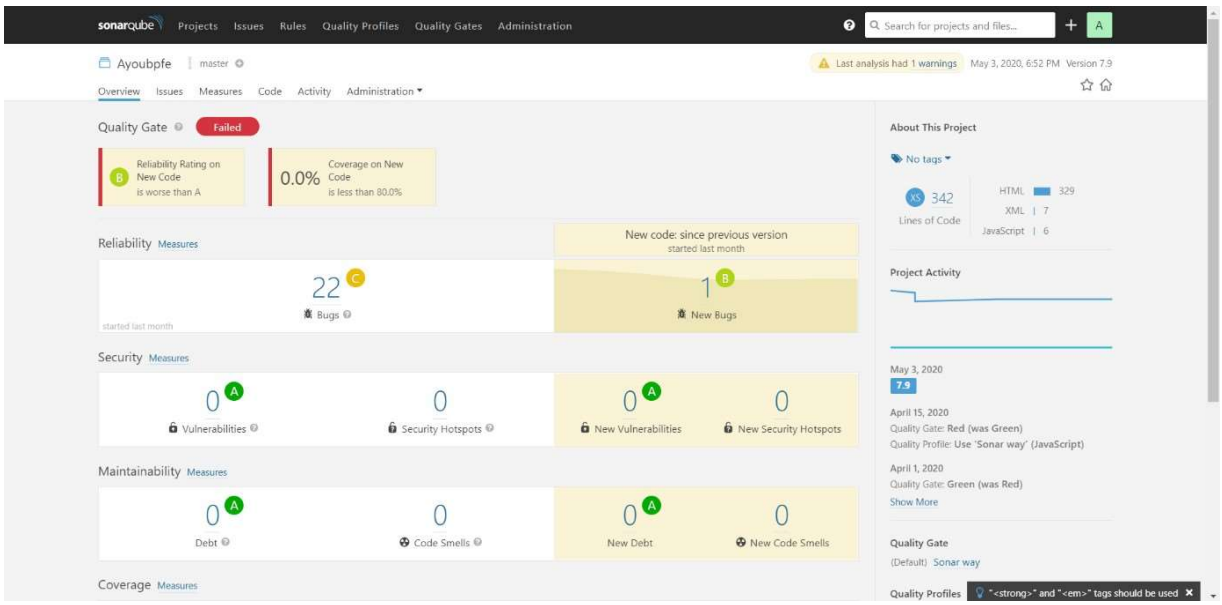


Figure 42-b : Résultat de la première analyse coté sonar

Nous constatons aussi qu'il y a des lignes de code qui manque une couverture, donc soit nous les donnerons un sens ou nous les supprimerons.

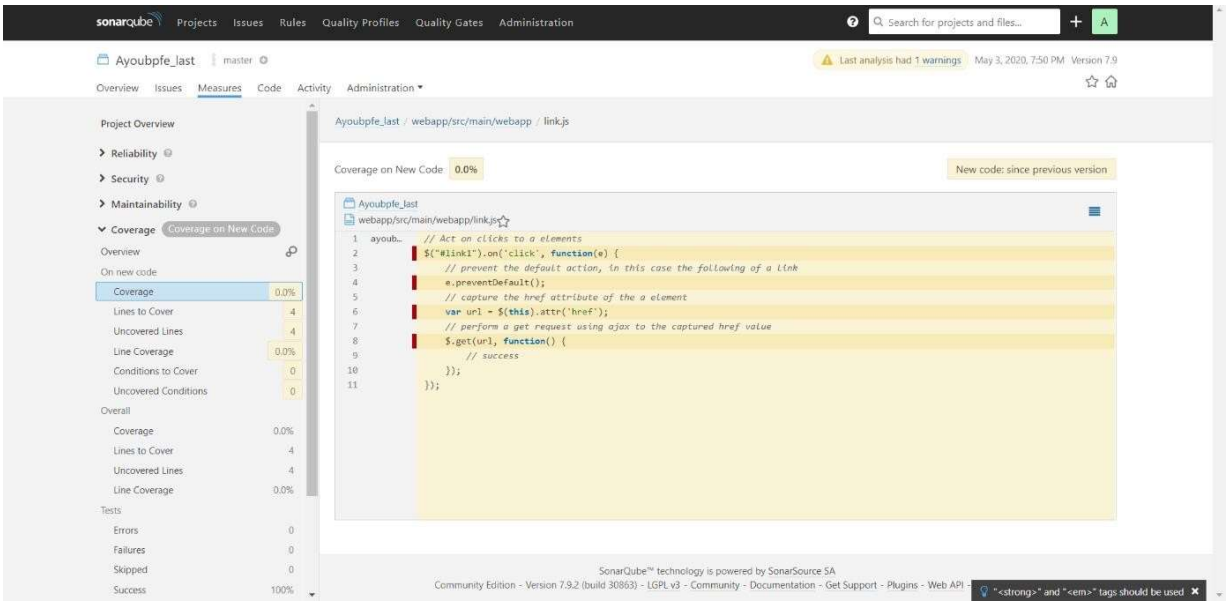


Figure 42-c : Suggestion de correction Sonar

Après avoir corrigé ces erreurs, nous relançons cette tâche :

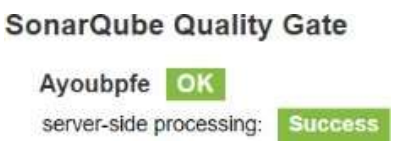


Figure 43 : Résultat du deuxième analyse Sonar coté Jenkins

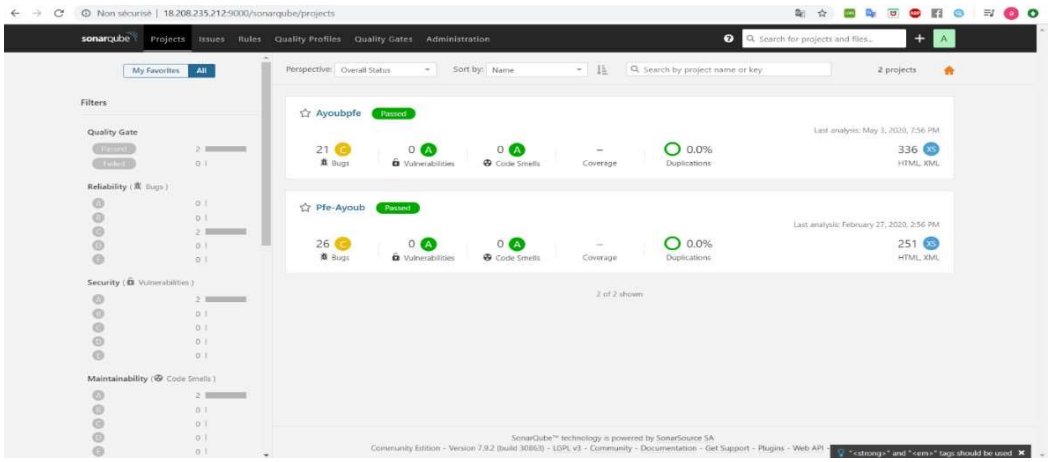


Figure 43-b : Résultat du deuxième analyse coté Sonar

3.4. Partie déploiement automatisé – Ansible :

Pour pouvoir mettre en place le serveur d'automatisation, nous devons suivre ces étapes :

1. Nous créons une nouvelle instance Amazon Linux avec le nom : Ansible-server.
2. Nous Installons Python et Ansible.
3. Ensuite, nous Ajoutons un utilisateur :
   #mkdir /etc/ansible
4. Nous éliminons l'exigence d'un mot de passe.
5. Après, nous modifions le compte d'utilisateur ansadmin et nous autorisons l'accès par mot de passe pour le service d'AWS EC2.
6. Nous générons une clé dans notre serveur pour pouvoir se connecter à Jenkins et lancer des tâches depuis ce serveur chaque fois.
7. Retour vers « ansible-mode-control », puis nous mettons la clé.
8. Nous créons un fichier contenant les adresses IP qu'Ansible va s'adresser.
9. Nous testons la connectivité avec nos serveurs en utilisant « ping » :

```
ansadmin@ansible-control-mode ansible]$ ansible all -m ping
WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python, but future installation of another Python interpreter could change this. See
ttps://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
ocalhost | SUCCESS => {
   "ansible_facts": {
       "discovered_interpreter_python": "/usr/bin/python"
   },
   "changed": false,
   "ping": "pong"

WARNING]: Platform linux on host 172.31.83.247 is using the discovered Python interpreter at /usr/bin/python, but future installation of another Python interpreter could change this.
ee https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
72.31.83.247 | SUCCESS => {
   "ansible_facts": {
       "discovered_interpreter_python": "/usr/bin/python"
   },
   "changed": false,
   "ping": "pong"

ansadmin@ansible-control-mode ansible]$
```

Figure 44 : Test Ping

10. Nous allons configurer la section « publish over SSH » et nous remplissons les champs qui manquent comme le montre la figure ci-dessous :

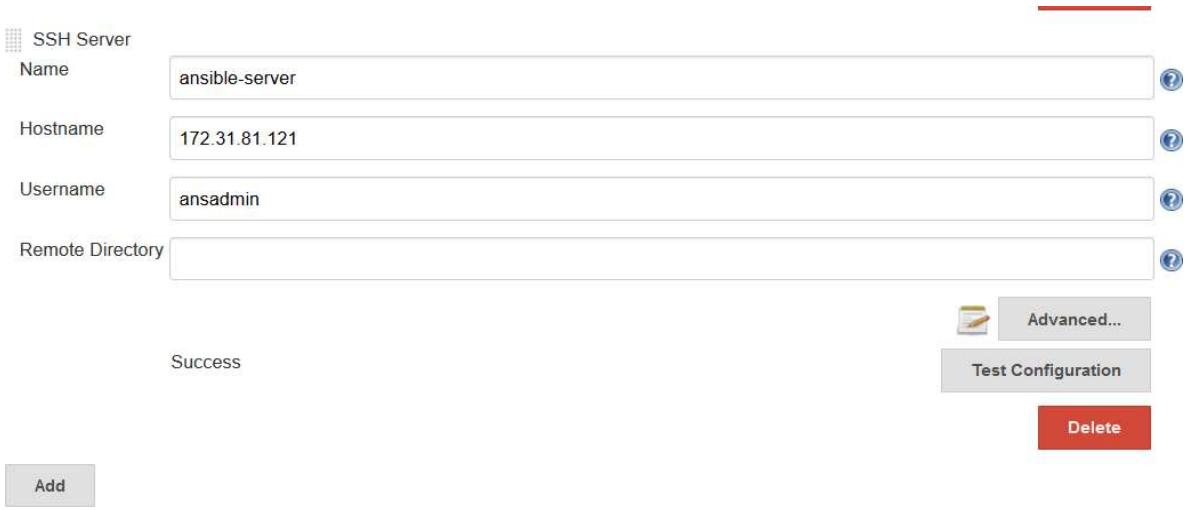


Figure 45 : Publish over SSH

3.5. Intégration de Docker Hub dans le pipeline :

Cette partie est consacrée pour stocker notre image dans Docker Hub et créer des conteneurs dans le serveur docker. Pour cela, nous devons suivre ces étapes :

1- Nous allons créer un compte sur Docker Hub

2- Nous allons tagger notre image avant de la pousser sur Docker Hub

3- Ensuite, Nous faisons l'authentification et puis nous poussons notre image…

```
[ansadmin@ip-172-31-81-121 docker]$ docker push ayoubbargaoui/simple-devops-image
The push refers to repository [docker.io/ayoubbargaoui/simple-devops-image]
6cbcb1da25e7: Pushed
faa4995d5333: Mounted from library/tomcat
c4652e217d57: Mounted from library/tomcat
```

Figure 46 : Push l'image vers Docker Hub

Vérification dans Docker Hub :

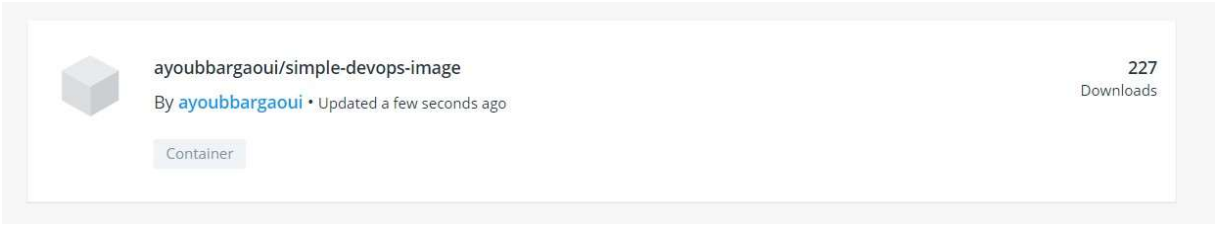


Figure 46-b : Répertoire Docker Hub

4- Enfin, nous supprimons l'image localement, pour que la prochaine fois, nous ne tirons l'image que depuis Docker hub.

3.5.1. Tagging l'image docker en utilisant Ansible playbook :

Dans cette partie, nous aurons 2 « Playbooks » :

- Le premier va être exécuté dans le serveur Ansible pour la création d'une image et la mettre dans Docker Hub, enfin il supprime cette image localement.
- Le deuxième va être exécuté sur le serveur Docker pour faire le « pull » de cette image et la création des conteneurs sur ce serveur, après vérification de l'état actuel de l'instance !

Le premier « Playbook »:

```
[ansadmin@ip-172-31-81-121 docker]$ vi create-simple-devops-image.yml
[ansadmin@ip-172-31-81-121 docker]$ cat create-simple-devops-image.yml
---
- hosts: all
  become: true
  tasks:

  - name: create docker image
    command: docker build -t simple-devops-image:latest .
    args:
      chdir: /opt/docker

  - name: create tag to image
    command: docker tag simple-devops-image:latest ayoubbargaoui/simple-devops-image

  - name: push image on to dockerhub
    command: docker push ayoubbargaoui/simple-devops-image

  - name: remove docker images from ansible server
    command: docker rmi simple-devops-image:latest ayoubbargaoui/simple-devops-image
    ignore_errors: yes

[ansadmin@ip-172-31-81-121 docker]$
```

Figure 47: Playbook 1 (Docker)

Exécution :

```
[ansadmin@ip-172-31-81-121 docker]$ ansible-playbook -i hosts create-simple-devops-image.yml

PLAY [all] *********************************************************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************************
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python, but future installation of another Python interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [create docker image] *****************************************************************************************************************************************************
changed: [localhost]

TASK [create tag to image] *****************************************************************************************************************************************************
changed: [localhost]

TASK [push image on to dockerhub] **********************************************************************************************************************************************
changed: [localhost]

TASK [remove docker images from ansible server] ********************************************************************************************************************************
changed: [localhost]

PLAY RECAP *********************************************************************************************************************************************************************
localhost                  : ok=5    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ansadmin@ip-172-31-81-121 docker]$
```

Figure 47-b : Exécution Playbook 1 (Docker)

Le deuxième « Playbook » :

```
[ansadmin@ip-172-31-81-121 docker]$ vi create-simple-devops-project.yml
[ansadmin@ip-172-31-81-121 docker]$ cat create-simple-devops-project.yml
---
- hosts: all
  become: true

  tasks:

  - name: stop if we have old docker container
    command: docker container stop simple-devops-container
    ignore_errors: yes

  - name: remove stopped docker container
    command: docker container rm simple-devops-container
    ignore_errors: yes

  - name: remove current docker image
    command: docker rmi ayoubbargaoui/simple-devops-image:latest
    ignore_errors: yes

#  - name: building docker image
#    command: docker build -t simple-devops-image .
#    args:
#      chdir: /opt/docker

  - name: pull docker image from dockerhub
    command: docker pull ayoubbargaoui/simple-devops-image:latest

  - name: create container using simple-devops-image
    command: docker run -d --name simple-devops-container -p 8080:8080 ayoubbargaoui/simple-devops-image:latest

[ansadmin@ip-172-31-81-121 docker]$
```

Figure 48 : Playbook 2 (Docker)

Nous exécutons ce Playbook:

```
[ansadmin@ip-172-31-81-121 docker]$ ansible-playbook -i hosts create-simple-devops-project.yml
[DEPRECATION WARNING]: The TRANSFORM_INVALID_GROUP_CHARS settings is set to allow bad characters in group names by default, this will change, but still be user configurable on
deprecation. This feature will be removed in version 2.10. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [docker-host] ********************************************************************************************************************************************************

TASK [Gathering Facts] ****************************************************************************************************************************************************
[WARNING]: Platform linux on host 172.31.83.247 is using the discovered Python interpreter at /usr/bin/python, but future installation of another Python interpreter could change this.
See https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
ok: [172.31.83.247]

TASK [stop if we have old docker container] *******************************************************************************************************************************
changed: [172.31.83.247]

TASK [remove stopped docker container] ************************************************************************************************************************************
changed: [172.31.83.247]

TASK [remove current docker image] ****************************************************************************************************************************************
changed: [172.31.83.247]

TASK [pull docker image from dockerhub] ***********************************************************************************************************************************
changed: [172.31.83.247]

TASK [create container using simple-devops-image] *************************************************************************************************************************
changed: [172.31.83.247]

PLAY RECAP ****************************************************************************************************************************************************************
172.31.83.247              : ok=6    changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ansadmin@ip-172-31-81-121 docker]$
```

Figure 48-b : Exécution Playbook 2 (Docker)

Ensuite, nous avons changé le fichier « hosts » et nous avons séparé les 2 adresses par un nom.

Nous pouvons appeler ces adresses IP par un nom au lieu de taper chaque fois l'adresse.

```
[ansadmin@ip-172-31-81-121 docker]$ cat hosts
[ansible]
localhost

[docker-host]
172.31.83.247
[ansadmin@ip-172-31-81-121 docker]$
```

Figure 49 : Fichier Hosts

3.5.2. Version finale - Job Jenkins :

1- Nous Créons un nouveau job sur Jenkins intitulé :
Deploy_on_Dockerhost_Container_using_Ansible

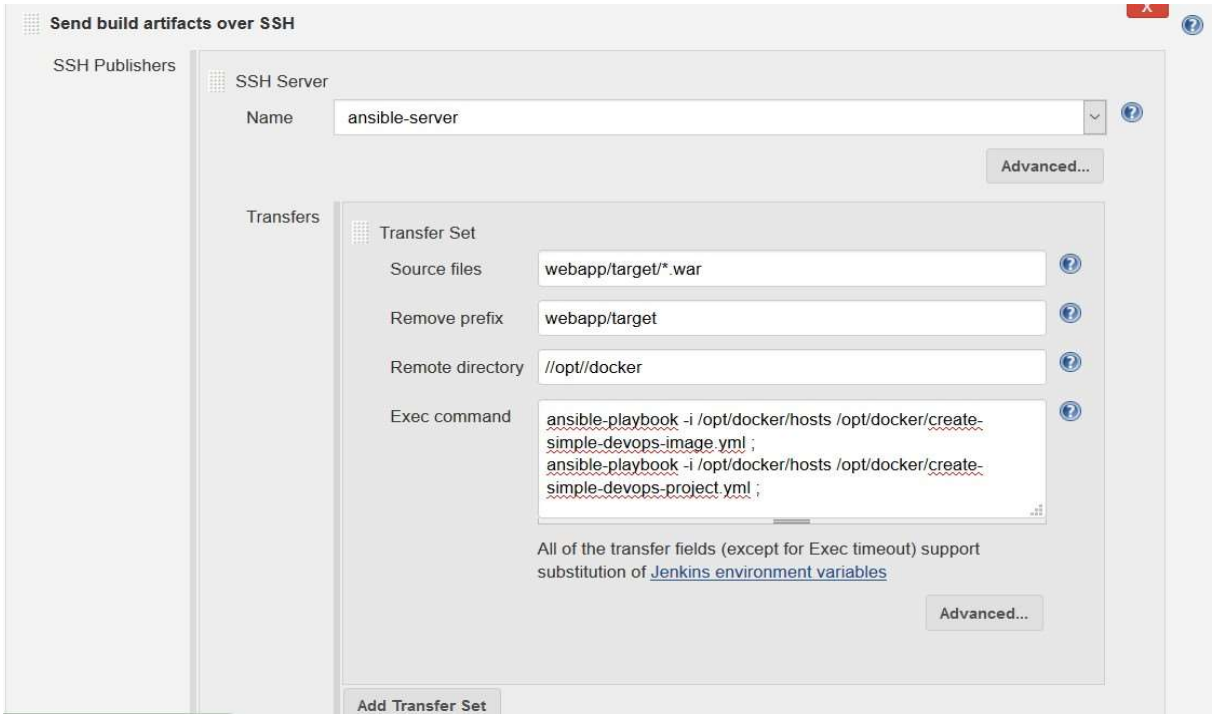


Figure 50 : Caractéristiques du job final (Docker)

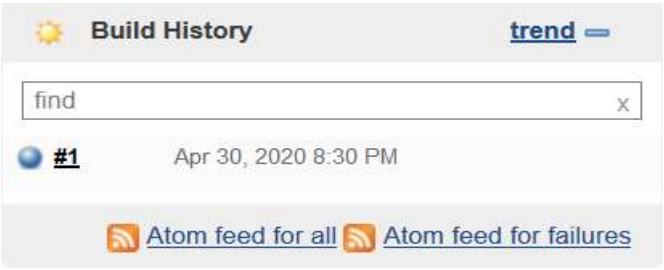


Figure 51 : Test final

Optionnel : Nous pouvons changer notre source code. Maintenant, automatiquement il y aura le redéploiement !

REMARQUES : Supposons que notre conteneur n'est pas en marche (exemple : nous avons fermé l'instance), nous devons le créer manuellement et après nous faisons la réexécution du job Jenkins mais le job Jenkins ne sera exécuté que s'il y a un nouveau « build ».

⇨ Pour dépasser ce problème, nous devons avoir quelques mécanismes qui prend soin de nos conteneurs.

⇨ Ici, c'est le rôle du Kubernetes entre en jeu…

3.6. Intégration de Kubernetes dans le pipeline :

1- Nous allons créer une instance Ubuntu 16.04 ou 18.04 (t2.medium) pour le master et deux autres instances (t2.micro, 30GB) pour les 2 autres nœuds récepteurs.

2- Après avoir installé kubeadm, kubelet, kubectl, kubernetes-cni dans les 3 nœuds :

```
root@ip-172-31-46-228:/home/ubuntu# apt-get install -y kubelet kubeadm kubectl kubernetes-cni
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  conntrack cri-tools ebtables socat
The following NEW packages will be installed:
  conntrack cri-tools ebtables kubeadm kubectl kubelet kubernetes-cni socat
0 upgraded, 8 newly installed, 0 to remove and 11 not upgraded.
Need to get 52.1 MB of archives.
After this operation, 275 MB of additional disk space will be used.
```

Figure 52 : Installation de kubeadm, kubelet, kubectl, kubernetes-cni

3- Maintenant, nous créons les « AMI » pour le nœud « master » :

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 172.31.46.228:6443 --token okdt32.o5ujafwgr5d145ao \
    --discovery-token-ca-cert-hash sha256:106d47fe01795127327b82155e41f1ce457dc0a5eba0c852612a7dac039c4814
root@ip-172-31-46-228:/home/ubuntu#
```

Figure 53: kubeadm init

Remarque :

Pour pouvoir exécuter les commandes kubectl qui nous permettent de suivre nos nœuds et « pods », nous devons copier le fichier de configuration « admin.conf » dans le dossier config que nous créons dans .kube !

```
root@ip-172-31-46-228:/home/ubuntu# kubectl get pods --all-namespaces
NAMESPACE     NAME                                          READY   STATUS    RESTARTS   AGE
kube-system   coredns-66bff467f8-54bzp                      0/1     Pending   0          3m20s
kube-system   coredns-66bff467f8-p4dl9                      0/1     Pending   0          3m20s
kube-system   etcd-ip-172-31-46-228                         1/1     Running   0          3m28s
kube-system   kube-apiserver-ip-172-31-46-228               1/1     Running   0          3m28s
kube-system   kube-controller-manager-ip-172-31-46-228      1/1     Running   0          3m28s
kube-system   kube-proxy-sfbdx                              1/1     Running   0          3m19s
kube-system   kube-scheduler-ip-172-31-46-228               1/1     Running   0          3m27s
root@ip-172-31-46-228:/home/ubuntu#
```

Figure 54 : Afficher les nœuds avec kubectl

3.6.1. Mettre en place le premier nœud :

```
root@ip-172-31-91-48:~# kubeadm join 172.31.46.228:6443 --token gbodnd.7w6y6pw3uya25a08 --discovery-token-ca-cert-hash sha256:106d47fe0179512732
4814
W0501 19:45:08.187765   18831 join.go:346] [preflight] WARNING: JoinControlPane.controlPlane settings will be ignored when control-plane flag is not set.
[preflight] Running pre-flight checks
        [WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the Docker cgroup driver. The recommended driver is "systemd". Please follow the guide at https://kubernetes.io/docs/setup
cri/
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -oyaml'
[kubelet-start] Downloading configuration for the kubelet from the "kubelet-config-1.18" ConfigMap in the kube-system namespace
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

root@ip-172-31-91-48:~#
```

```
root@ip-172-31-46-228:/home/ubuntu# kubectl get nodes
NAME               STATUS   ROLES    AGE   VERSION
ip-172-31-46-228   Ready    master   24m   v1.18.2
ip-172-31-91-48    Ready    <none>   88s   v1.17.3
root@ip-172-31-46-228:/home/ubuntu#
```

Figure 55 : Mettre en place le premier nœud

3.6.2. Mettre en place le deuxième nœud :

```
ubuntu@ip-172-31-93-131:~$ sudo su -
root@ip-172-31-93-131:~# kubeadm join 172.31.46.228:6443 --token gbodnd.7w6y6pw3uya25a08 --discovery-token-ca-cert-hash sha256:106d47fe0179512732
c4814
W0501 19:50:33.147692    4551 join.go:346] [preflight] WARNING: JoinControlPane.controlPlane settings will be ignored when control-plane flag is not set.
[preflight] Running pre-flight checks
        [WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the Docker cgroup driver. The recommended driver is "systemd". Please follow the guide at https://kubernetes.io/docs/setu
cri/
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -oyaml'
[kubelet-start] Downloading configuration for the kubelet from the "kubelet-config-1.18" ConfigMap in the kube-system namespace
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

root@ip-172-31-93-131:~#
```

```
root@ip-172-31-46-228:/home/ubuntu# kubectl get nodes
NAME               STATUS   ROLES    AGE    VERSION
ip-172-31-46-228   Ready    master   28m    v1.18.2
ip-172-31-91-48    Ready    <none>   6m4s   v1.17.3
ip-172-31-93-131   Ready    <none>   38s    v1.17.3
root@ip-172-31-46-228:/home/ubuntu#
```

Figure 56 : Mettre en place le deuxième nœud

3.6.3. Création des fichiers YAML de déploiement et service :

Il s'agit de deux scripts, un consacré pour le déploiement et l'autre un service pour l'accès utilisateur. Le script de déploiement est un fichier YAML où nous pouvons préciser plusieurs paramètres. Dans la figure ci-dessous :

- Un déploiement nommé pfe-deployment est créé, indiqué par le champ : .metadata.name.
- Le déploiement crée deux pods répliqués, indiqués par le champ replicas.

- Le champ selector définit comment le déploiement trouve les pods à gérer. Dans ce cas, vous sélectionnez simplement un label définie dans le template de pod (app :pfe-devops-project). Cependant, des règles de sélection plus sophistiquées sont possibles, tant que le modèle de pod satisfait lui-même la règle.
- Le port que prend les conteneurs de déploiement est 8080, et ces derniers pointent vers l'image docker dans le répertoire dans Docker Hub indiqué dans le champ « image ».

```
root@ip-172-31-46-228:/home/ubuntu# vi pfe-deploy.yml
root@ip-172-31-46-228:/home/ubuntu# cat pfe-deploy.yml
apiVersion: apps/v1 # for versions before 1.9.0 use apps/v1beta2
kind: Deployment
metadata:
  name: pfe-deployment
spec:
  selector:
    matchLabels:
      app: pfe-devops-project
  replicas: 2 # tells deployment to run 2 pods matching the template
  strategy:
   type: RollingUpdate
   rollingUpdate:
     maxSurge: 1
     maxUnavailable: 1

  template:
    metadata:
      labels:
        app: pfe-devops-project
    spec:
      containers:
      - name: pfe-devops-project
        image: ayoubbargaoui/simple-devops-image
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
root@ip-172-31-46-228:/home/ubuntu#
```

Figure 57 : Script YAML de déploiement

Le script de la figure dans la page suivante est pour le service (dans Kubernetes un service est un objet REST, semblable à un pod. Comme tous les objets REST, vous pouvez effectuer un POST d'une définition de service sur le serveur API pour créer une nouvelle instance.), dans notre cas cette spécification crée un nouvel objet Service nommé « pfe-service », qui cible le port TCP 31200 sur n'importe quel pod avec l'étiquette « app=pfe-devops-project ».

```
root@ip-172-31-46-228:/home/ubuntu# vi pfe-service.yml
root@ip-172-31-46-228:/home/ubuntu# cat pfe-service.yml
apiVersion: v1
kind: Service
metadata:
  name: pfe-service
  labels:
    app: pfe-devops-project
spec:
  selector:
    app: pfe-devops-project
  type: LoadBalancer
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 31200
root@ip-172-31-46-228:/home/ubuntu#
```

Figure 58 : Script YAML de service

Vérification après exécution :

```
root@ip-172-31-46-228:/home/ubuntu# ls
pfe-deploy.yml  pfe-service.yml
root@ip-172-31-46-228:/home/ubuntu# kubectl apply -f /home/ubuntu/pfe-deploy.yml
deployment.apps/pfe-deployment created
root@ip-172-31-46-228:/home/ubuntu# kubectl get nodes
NAME               STATUS   ROLES    AGE    VERSION
ip-172-31-46-228   Ready    master   147m   v1.18.2
ip-172-31-91-48    Ready    <none>   124m   v1.17.3
ip-172-31-93-131   Ready    <none>   119m   v1.17.3
root@ip-172-31-46-228:/home/ubuntu# kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
pfe-deployment-86cc6b957-65sjs    1/1     Running   0          26s
pfe-deployment-86cc6b957-rsnc9    1/1     Running   0          26s
root@ip-172-31-46-228:/home/ubuntu# kubectl apply -f /home/ubuntu/pfe-service.yml
service/pfe-service created
root@ip-172-31-46-228:/home/ubuntu# kubectl get deploy
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
pfe-deployment   2/2     2            2           43s
root@ip-172-31-46-228:/home/ubuntu# kubectl get service
NAME          TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
kubernetes    ClusterIP      10.96.0.1        <none>        443/TCP          147m
pfe-service   LoadBalancer   10.108.210.121   <pending>     8080:31200/TCP   13s
root@ip-172-31-46-228:/home/ubuntu#
```

Figure 59 : Vérification après exécution de deploy et service

3.6.4. Intégration de Kubernetes avec Ansible :

1- Comme d'habitude, nous effectuons un échange de clés et l'authentification SSH entre les 2 serveurs, puis sous le chemin /opt/ dans le serveur Ansible, nous créons un dossier Kubernetes qui va contenir les « playbooks » à exécuter :

```
[ansadmin@ip-172-31-81-121 .ssh]$ cd /opt/
[ansadmin@ip-172-31-81-121 opt]$ ls
aws  containerd  docker
[ansadmin@ip-172-31-81-121 opt]$ sudo mkdir kubernetes
[ansadmin@ip-172-31-81-121 opt]$ ls
aws  containerd  docker  kubernetes
[ansadmin@ip-172-31-81-121 opt]$
```

Figure 60: /opt/Kubernetes/

2- Nous modifions le fichier Hosts :

```
[ansadmin@ip-172-31-81-121 kubernetes]$ sudo vi hosts
[ansadmin@ip-172-31-81-121 kubernetes]$ cat hosts
[ansible-server]
localhost

[kubernetes]
172.31.46.228

[ansadmin@ip-172-31-81-121 kubernetes]$
```

Figure 61 : fichier Hosts

Nous allons créer le 1er playbook qui va réagir avec « pfe-deploy.yml » qui se trouve dans l'instance « master k8s », nommé : « deployment-pfe.yml » :

```
[ansadmin@ip-172-31-81-121 kubernetes]$ sudo vi deployment-pfe.yml
[ansadmin@ip-172-31-81-121 kubernetes]$ cat deployment-pfe.yml
---
- name: Create pods using deployment
  hosts: kubernetes
  become: true
  user: root

  tasks:
  - name: create a deployment
    command: kubectl apply -f /home/ubuntu/pfe-deploy.yml

  - name: update deployment with new pods if image updated in docker hub
    command: kubectl rollout restart deployment.v1.apps/pfe-deployment
[ansadmin@ip-172-31-81-121 kubernetes]$
```

Figure 62 : Playbook "deployment"

Nous allons créer le deuxième « playbook » qui va réagir avec « pfe-service.yml » qui se trouve dans l'instance « k8s-master », nommé : service-pfe.yml

```
[ansadmin@ip-172-31-81-121 kubernetes]$ sudo vi service-pfe.yml
[ansadmin@ip-172-31-81-121 kubernetes]$ cat service-pfe.yml
---
- name: create service for deployment
  hosts: kubernetes
  become: true
  user: root

  tasks:
  - name: create a service
    command: kubectl apply -f /home/ubuntu/pfe-service.yml
[ansadmin@ip-172-31-81-121 kubernetes]$
```

Figure 63: Playbook "service"

Nous exécutons ces 2 « playbooks » :

```
[ansadmin@ip-172-31-81-121 kubernetes]$ ansible-playbook -i hosts deployment-pfe.yml
[DEPRECATION WARNING]: The TRANSFORM_INVALID_GROUP_CHARS settings is set to allow bad characters in group names by default, this will change, but still be user configurable on
deprecation. This feature will be removed in version 2.10. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [Create pods using deployment] ********************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************
ok: [172.31.46.228]

TASK [create a deployment] *****************************************************************************************************************************************
changed: [172.31.46.228]

TASK [update deployment with new pods if image updated in docker hub] **********************************************************************************************
changed: [172.31.46.228]

PLAY RECAP *********************************************************************************************************************************************************
172.31.46.228              : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
[ansadmin@ip-172-31-81-121 kubernetes]$ ansible-playbook -i hosts service-pfe.yml
[DEPRECATION WARNING]: The TRANSFORM_INVALID_GROUP_CHARS settings is set to allow bad characters in group names by default, this will change, but still be user configurable on
deprecation. This feature will be removed in version 2.10. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [create service for deployment] *******************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************
ok: [172.31.46.228]

TASK [create a service] ********************************************************************************************************************************************
changed: [172.31.46.228]

PLAY RECAP *********************************************************************************************************************************************************
172.31.46.228              : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ansadmin@ip-172-31-81-121 kubernetes]$
```

Figure 64 : Exécution playbooks

Dans la partie suivante, nous allons créer 2 « jobs » dans Jenkins résumer tout ce travail d'une façon plus organisée et simple pour l'utilisateur. Pour cela, il y aura un « job » CI pour la création d'une image localement et déploiement dans Docker Hub. Et un autre « job » CD pour le déploiement dans nos nœuds Kubernetes en utilisant ces 2 Playbooks.

3.6.5. CI Job dans Jenkins pour créer une image Docker :

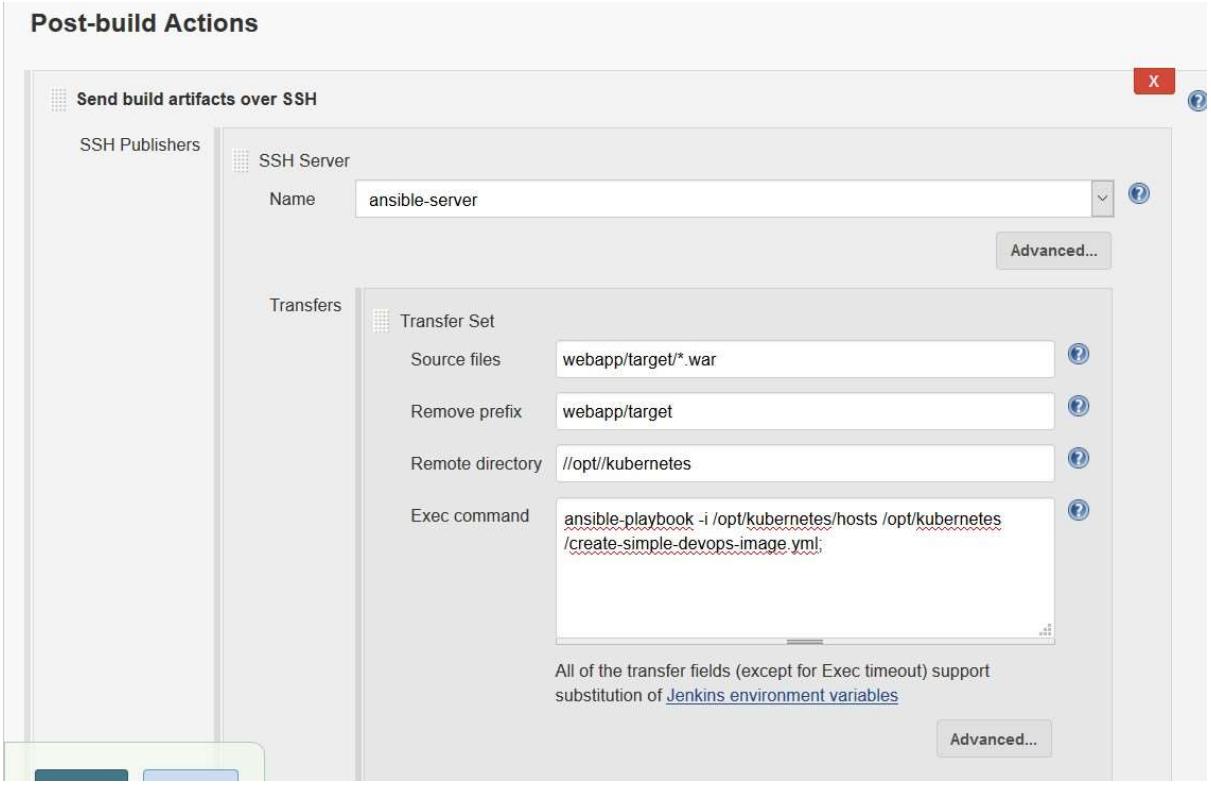


Figure 65 : CI job K8S

Remarque :

Nous allons ajouter une nouvelle stratégie :

Si ce job est valide, il déclenche un autre job qui est le CD job AUTOMATIQUEMENT !





Figure 65-b : Résultat CI job

Et à son tour : le CD job s'exécute automatiquement :

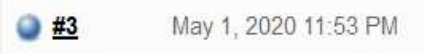


Figure 75-c : CD job s'exécute automatiquement

3.6.6. CD Job dans Jenkins pour déployer dans Kubernetes :

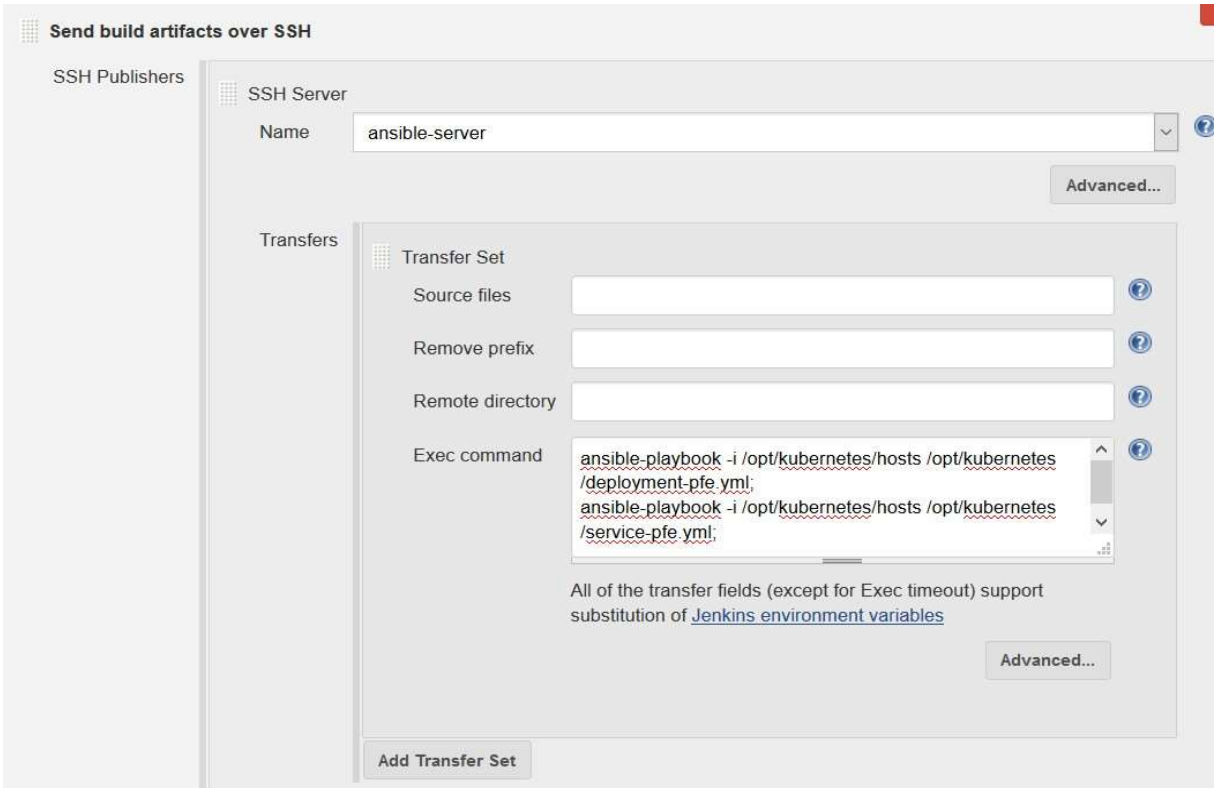


Figure 66 : CD Job K8S

Résultat :




```
PLAY RECAP *********************************************************************
172.31.46.228              : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

SSH: EXEC: completed after 6,408 ms
SSH: Disconnecting configuration [ansible-server] ...
SSH: Transferred 0 file(s)
Finished: SUCCESS
```

Figure 66-b : Résultat CD job

3.7. Interfaces web :

La dernière étape de ce projet, nous avons développé des interfaces web pour :

- Simplifier l'accès à nos serveurs en une seule clique sans avoir besoin de connaissances préalables dans le cloud.
- Exécuter des taches à distances sans avoir besoin chaque fois d'ouvrir le serveur Jenkins, mais seulement en une seule clique.
- Modifier les scripts de configuration avec un outil graphique.

Vous trouvez ci-dessous les images de nos interfaces :

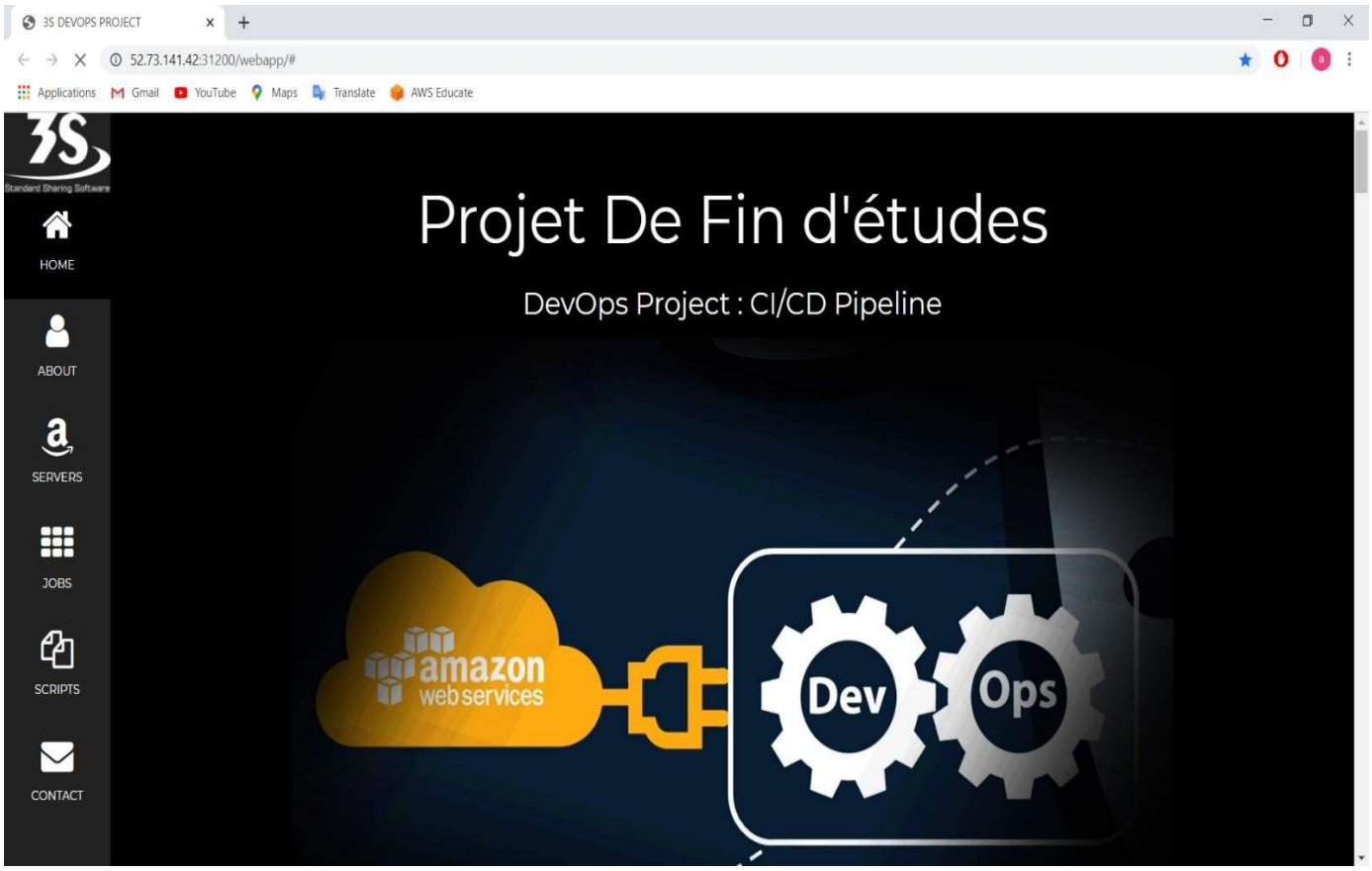


Figure 67 : Page d'accueil

Dans la page suivante, comme vous pouvez remarquer d'après la figure, nous avons développé une page consacrée pour l'accès direct à nos serveurs :

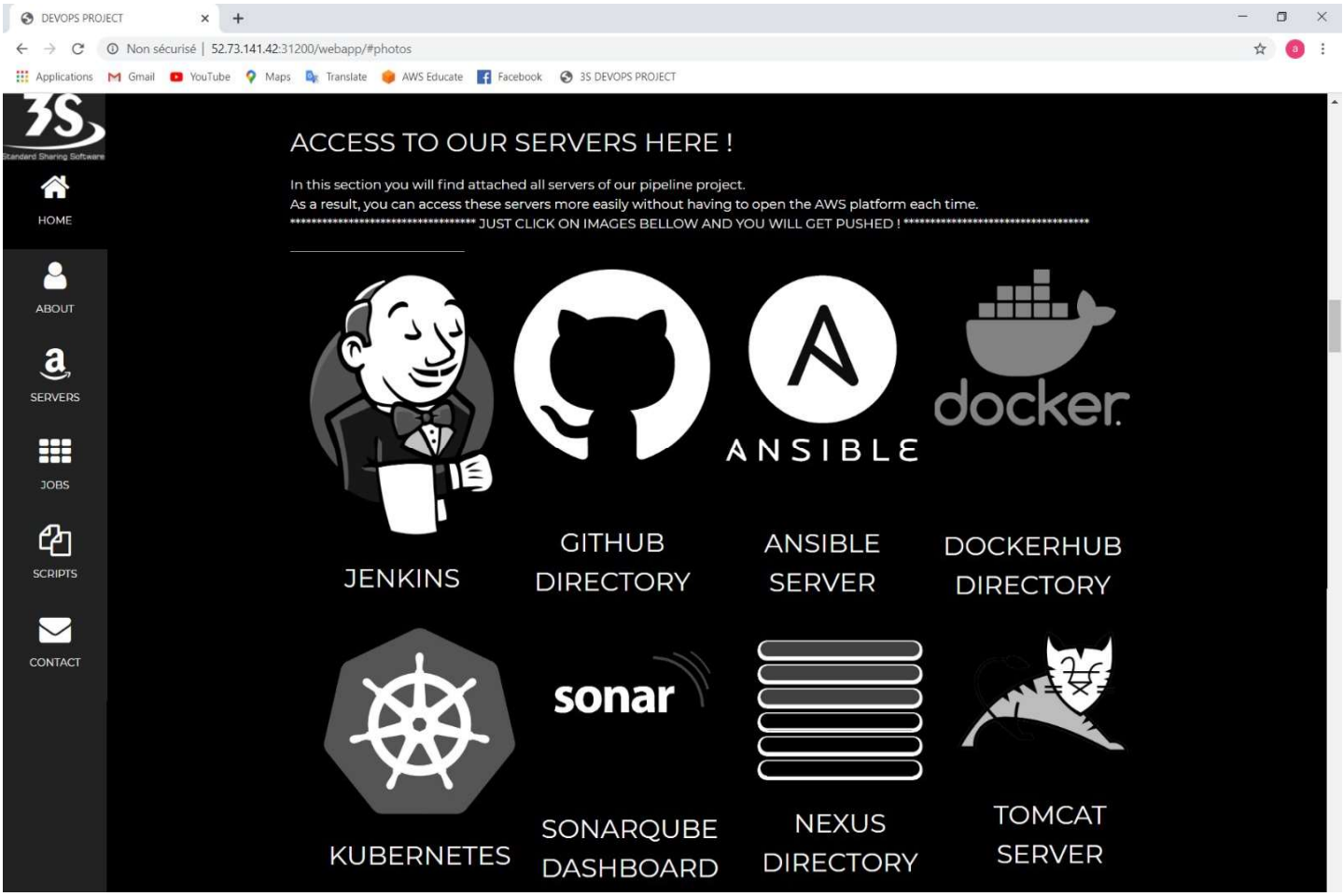


Figure 68 : Page d'accès aux serveurs

Fonctionnement :

- Nous cliquons sur chaque logo d'un serveur pour que nous serons poussés.

En plus :

- Si nous cliquons sur le logo de GitHub : Nous serons poussés vers le répertoire de notre application.
- Si nous cliquez sur le logo de Docker : nous accéderons vers notre répertoire Docker Hub où il y a vos images Docker.

Remarque :

- Il ne faut pas oublier de s'authentifier aux serveurs et aux répertoires…
- Dans la page suivante (Exécution des « jobs »), nous avons utilisé des API Jenkins pour pouvoir les exécuter à distance.

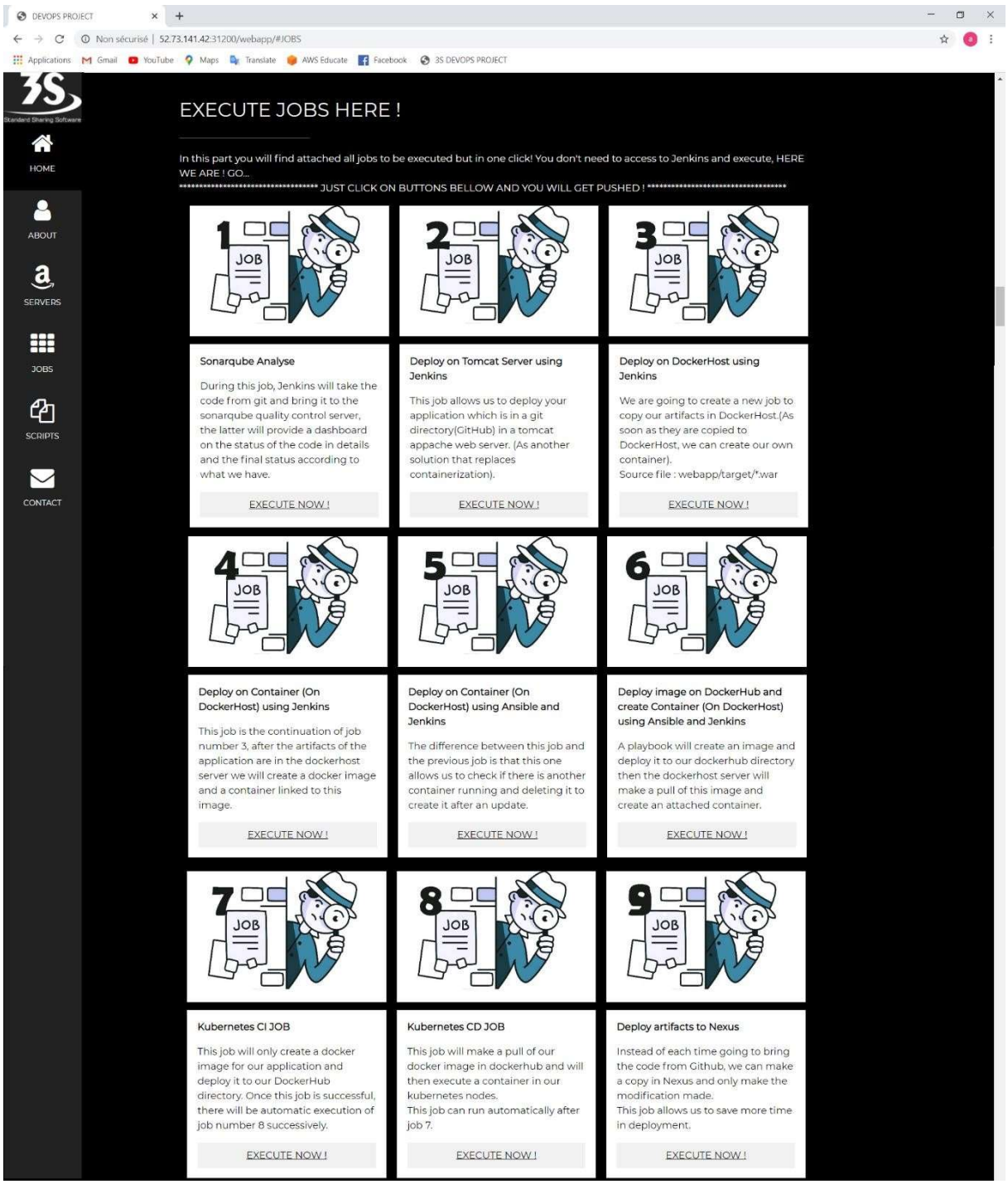


Figure 69 : Page d'exécution des « jobs »

D'après cette figure ci-dessus, nous pouvons maintenant exécuter nos « jobs » à distance sans accéder à notre serveur Jenkins, mais seulement de lire la description de chaque « job » et en une seule clique de les exécuter.

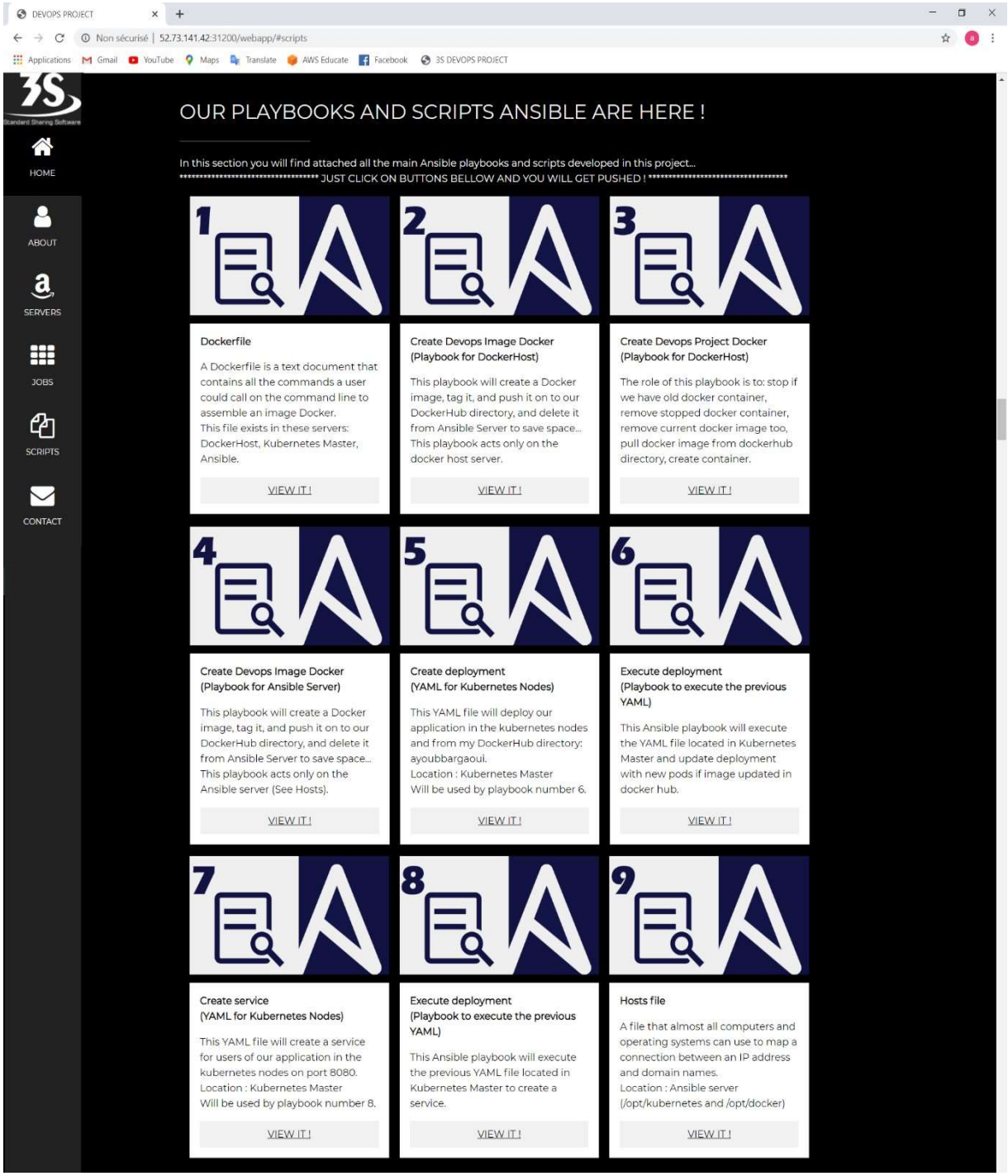


Figure 70 : Page d'accès aux scripts

D'après cette interface dans la figure ci-dessus, nous pouvons accéder à nos scripts en ligne et nous pouvons les modifier de façon collective avec les membres de l'équipe.

Finalement, dans la page suivante, nous finissons par une interface « contact » pour envoyer les « feedbacks » entre les membres.

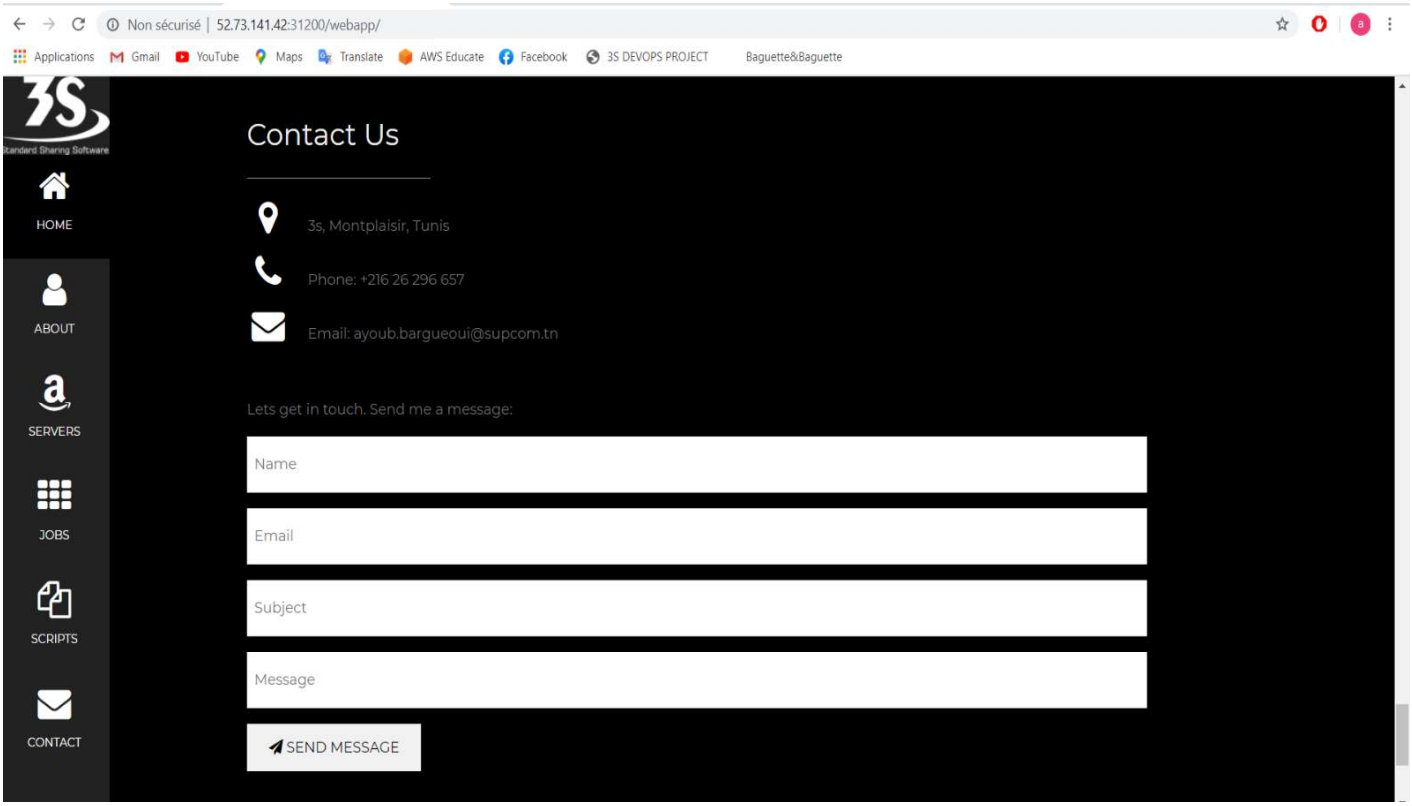


Figure 71 : Page contact

4. Conclusion :

Durant ce chapitre, nous avons vu la partie finale de ce projet, la réalisation, qui nous a abouti finalement à un site web qui gère ce pipeline implémenté dans le cloud AWS. Ainsi, nous avons vu comment mettre en place nos serveurs, puis nous avons expliqué nos scripts d'automatisation et les tâches dans Jenkins pour simplifier l'exécution de ces « jobs » et nous avons exposé les interfaces de notre site web.

A partir de ce moment, chaque application subit un cycle de vie complet et rapide, de la phase d'intégration continue (Tests, « Build », …) jusqu'à la phase de déploiement automatique qui facilite la mise en production de cette application le plus tôt possible.

CONCLUSION GENERALE

Dans ce projet de fin d'études, Nous étions amenés à mettre en place un pipeline d'intégration continue et de déploiement continue pour les applications web multi-services. Dès maintenant, il est donc possible d'observer l'évolution des applications durant tout un cycle d'une façon continue. Il est ainsi possible de découvrir plus rapidement d'éventuels erreurs afin d'y corriger dès leur apparition.

Le choix de ces outils en particulier, est une parmi plusieurs solutions qui existent dans le marché, c'est pour cela on a procédé à une étude comparative entre ces outils. Et pour les mettre en place, nous avons dégagé les besoins fonctionnels et non fonctionnels et nous avons fait la conception en des diagrammes UML avant d'entamer la partie implémentation.

Néanmoins, lors de ce travail, nous avons rencontré des problèmes liés aux ressources budgétaires sur le cloud AWS. En effet, nous étions devant l'obligation de réduire notre utilisation d'Amazon Elastic Compute Cloud (Amazon EC2) en arrêtant et en démarrant nos instances à des heures prédéfinies manuellement.

Nous proposons, comme perspective à ce travail, l'intégration des processus de surveillance et de mesure des activités des machines, c'est ce qu'on appelle souvent : Le monitoring. Il a pour objectif de suivre différents paramètres : performance du système, temps de réponse des interfaces, etc…

Finalement, ce projet de fin d'études est une belle conclusion de ces trois années passées au sein de Sup'Com. J'ai élargi mon spectre à d'autres compétences informatiques, grâce à tout ce que j'ai appris, ce qui peut être un atout à l'avenir si de futures missions autour de ce domaine me sont confiées.

ANNEXES

Préliminaires :

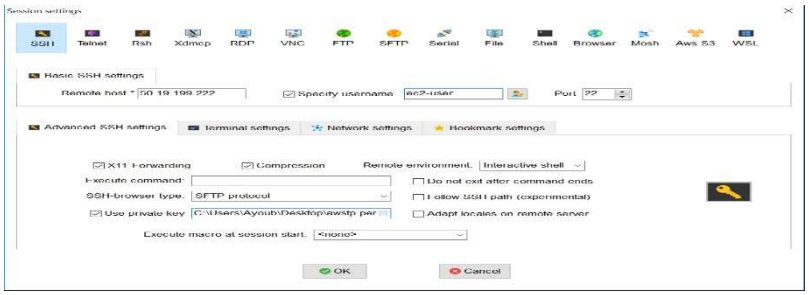
Après avoir lancé une instance, nous devons se connecter à celle-ci en utilisant le protocole SSH et une clé privée (.pem) créée par le fournisseur lui-même.

Name	Instance ID	Instance Type	Availability Zone	Instance State	Status Checks	Alarm Status	Public DNS (IPv4)	IPv4 Public IP
Jenkins_Server	i-01730c190a4073577	t2.micro	us-east-1d	running	2/2 checks ...	None	ec2-50-19-199-222.co...	50.19.199.222

Puis il existe 2 méthodes pour y accéder, soit nous tapons une commande ssh dans une invite de commande ou bien nous utilisons un logiciel de connexion à distance comme Putty ou MobaXterm.

```
C:\Users\Ayoub\Desktop>ssh -i "awstp.pem" ec2-user@ec2-50-19-199-222.compute-1.amazonaws.com
```

Dans notre cas, nous avons utilisé souvent le fameux logiciel MobaXterm, car il présente beaucoup d'options de plus par rapport aux autres tel que le lancement de plusieurs instances en parallèle.



Connexion avec MobaXterm

Partie Nexus :

Nexus est un référentiel d'artefacts qui aide à stocker les résultats de votre génération.

Conditions préalables :

Instance EC2 avec java

Étapes de mise en œuvre

Télécharger et configurer la version stable de Nexus

#cd / opt

#wget https://sonatype-download.global.ssl.fastly.net/nexus/3/nexus-3.0.2-02-unix.tar.gz

#tar -zxvf nexus-3.0.2-02-unix.tar.gz

#mv /opt/nexus-3.0.2-02 / opt / nexus

En tant que bonne pratique de sécurité, il n'est pas conseillé d'exécuter le service Nexus en tant que root. Créez donc un nouvel utilisateur appelé nexus et accordez à sudo l'accès à la gestion des services nexus

#sudo adduser nexus

#visudo \\ nexus ALL = (ALL) NOPASSWD: ALL

#sudo chown -R nexus: nexus / opt / nexus

Ouvrir le fichier /opt/nexus/bin/nexus.rc, décommentez le paramètre run_as_user et définissez-le comme suit.

#vi /opt/nexus/bin/nexus.rc

#run_as_user = "nexus" (le fichier ne doit avoir que cette ligne)

Ajouter Nexus en tant que service au démarrage

#sudo ln -s / opt / nexus / bin / nexus /etc/init.d/nexus

#service nexus start

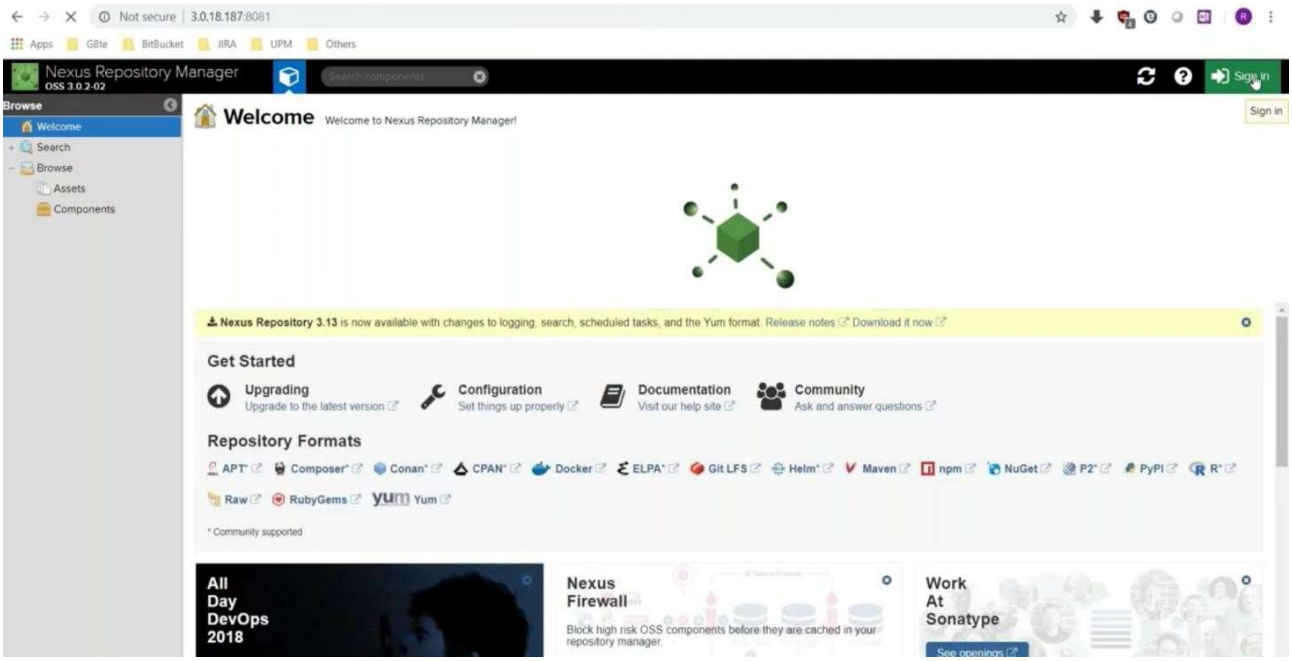
Connectez-vous au serveur Nexus à partir du navigateur sur le port 8081

http :// <Nexus_server>: 8081

Utiliser les informations d'identification par défaut pour se connecter :

Nom d'utilisateur : admin

Mot de passe : admin123



Nexus Server

Déployer une application dans un conteneur :

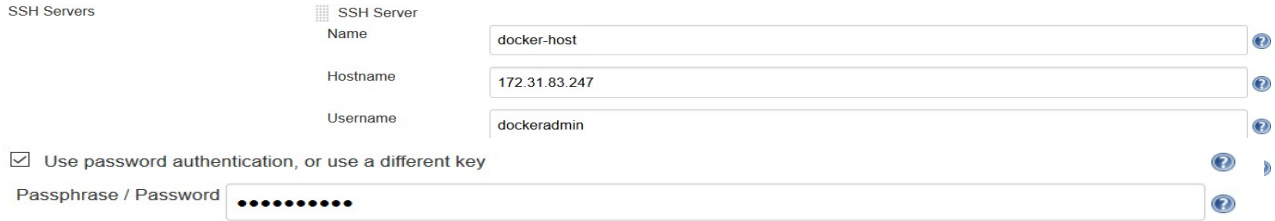
Après avoir installé Docker dans notre instance, Nous lançons le service comme le montre cette figure :

```
[root@docker ~]# service docker start
Starting cgconfig service:                                 [  OK  ]
Starting docker:        .                                  [  OK  ]
[root@docker ~]#
```

Puis on doit intégrer Docker avec Jenkins, donc on ajoute les plugins nécessaires :



Après la création d'un utilisateur, nous configurons la connexion ssh entre les 2 serveurs (Jenkins et l'utilisateur dockeradmin) :



Connexion SSH entre Jenkins et DockerHost

Nous autorisons l'authentification par mot de passe et nous relançons le serveur :

```
# EC2 uses keys for remote access
PasswordAuthentication yes
#PermitEmptyPasswords no
```

```
[root@docker ~]# vi /etc/ssh/sshd_config
[root@docker ~]# service sshd reload
Reloading sshd:                                            [  OK  ]
[root@docker ~]#
```

Relancer le serveur Docker

Maintenant nous créons une tâche qui construit une image Docker et exécute un conteneur lié à cette image. Pour cela, nous devons créer un fichier appelé : Dockerfile, c'est un fichier texte qui contient toutes les commandes qu'un utilisateur peut appeler sur la ligne de commande pour assembler une image.

```
From tomcat:latest

# Maintainer
MAINTAINER "Ayoub"

# copy war file on to container
COPY ./webapp.war /usr/local/tomcat/webapps
~
```

Fichier Dockerfile

Ce fichier doit appartenir au dossier qui contient l'application (Fichier .war) :

```
[dockeradmin@ip-172-31-83-247 ~]$ ls
Dockerfile  webapp.war
[dockeradmin@ip-172-31-83-247 ~]$
```

Nous construisons l'image manuellement en utilisant une commande Docker comme le montre la figure ci-dessous :

```
[dockeradmin@ip-172-31-83-247 ~]$ docker build -t devops-project .
Sending build context to Docker daemon     449kB
Step 1/3 : FROM tomcat:latest
 ---> 927899a31456
Step 2/3 : MAINTAINER AR ayoub
 ---> Running in cc663976754f
Removing intermediate container cc663976754f
 ---> b69279e0f37d
Step 3/3 : COPY  ./webapp.war /usr/local/tomcat/webapp
 ---> 744a3b464f3b
Successfully built 744a3b464f3b
Successfully tagged devops-project:latest
```

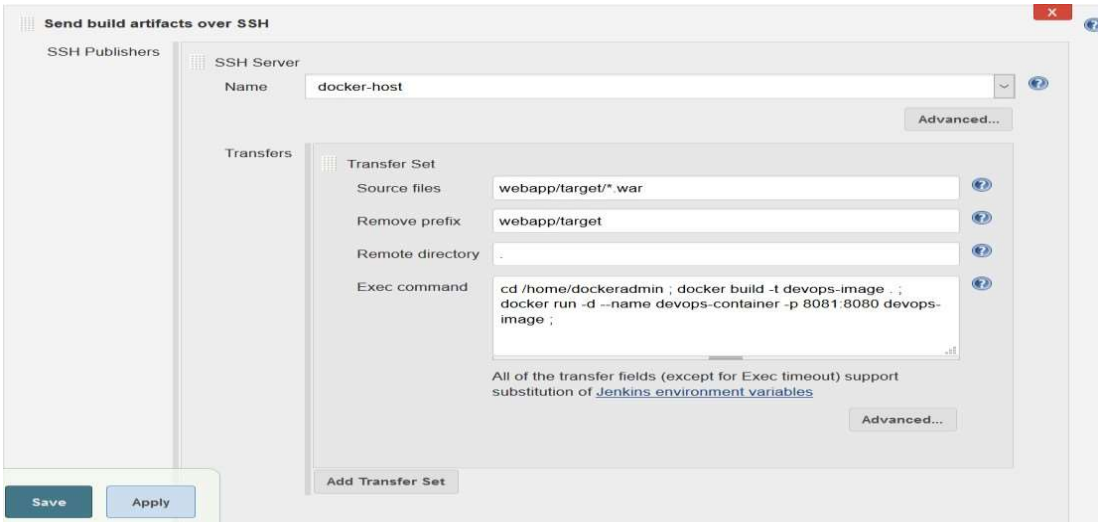
Création d'image Docker

Puis nous exécutons un conteneur lié à cette image :

```
[dockeradmin@ip-172-31-83-247 ~]$ docker run -d --name devops-container -p 8081:8080 devops-project
9026b6e69b6915e4e45411984a24ebde2c657dce45508269c3fdd2ab3b38ae7d
[dockeradmin@ip-172-31-83-247 ~]$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS                    NAMES
9026b6e69b69        devops-project      "catalina.sh run"   5 seconds ago       Up 3 seconds        0.0.0.0:8081->8080/tcp   devops-container
[dockeradmin@ip-172-31-83-247 ~]$
```

Exécution du conteneur

Au lieu de les lancer manuellement, nous allons créer un job qui exécute ces 2 commandes en parallèle, ce job appelé : Deploy_on_Container :



Nous pouvons vérifier la création du conteneur en tapant l'adresse ip public dans le navigateur sur le port 8081.

--.....FIN ANNEXES.....

LISTE DES RÉFÉRENCES

[1] 3s. [En ligne]. Available: https://www.3s.com.tn/.

[2] syloe, syloe, [En ligne]. Available: https://www.syloe.com/glossaire/approche-devops/.

[3] Git. [En ligne]. Available: https://git-scm.com/.

[4] Jenkins, «jenkins.io,» [En ligne]. Available: https://www.jenkins.io/doc/book/.

[5] sonarqube, sonar, [En ligne]. Available: https://docs.sonarqube.org/latest/.

[6] Y. A. w. Scripting, «Automation with Scripting,» [En ligne]. Available: https://www.youtube.com/watch?v=sGeZm pZ9I&list=PL2qzCKTbjutIyQAe3GglWISLnLTQLGm7e.

[7] S. Hykes, «Wikipedia,» Docker, [En ligne]. Available: https://fr.wikipedia.org/wiki/Docker_(logiciel).

[8] Kubernetes, «kubernetes.io,» Google, [En ligne]. Available: https://kubernetes.io/fr/docs/home/.

[9] L. Grangeau, Openclassroom, [En ligne]. Available: https://openclassrooms.com/fr/courses/2035736-mettez-en-place-lintegration-et-la-livraison-continues-avec-la-demarche-devops/6182691-quest-ce-que-lintegration-continue.

[10] «Les étapes de la CI,» [En ligne]. Available: http://igm.univ-mlv.fr/~dr/XPOSE2012/Integration%20Continue/concept.html.

[11] Coursera, [En ligne]. Available: https://fr.coursera.org/.

[12] Amazon AWS, [En ligne]. Available: https://aws.amazon.com/fr/devops/continuous-delivery/#:~:text=Explications%20%C3%A0%20propos%20de%20la,dans%20un%20environnement%20de%20production..

[13] [En ligne]. Available: http://www.alcantis.fr/index_fichiers/virtualisation_systemes_information.pdf.

[14] «Git,» [En ligne]. Available: https://git-scm.com/doc/ext.

[15] «Ansible,» [En ligne]. Available: https://docs.ansible.com/.

RÉSUMÉ :

L'objectif de ce projet est de mettre en place une plateforme d'intégration continue et de livraison et déploiement continu pour les applications web multi-services. Après une étude des principaux concepts et pratiques de DevOps, nous avons effectué une étude comparative des outils de ces pratiques. Suite à cette étude comparative, une analyse des besoins et une conception de l'architecture du projet se sont apparues nécessaires, avant l'implémentation de ce pipeline CI/CD. Ce projet a permis de minimiser le temps de production et d'organiser le cycle de vie de chaque application. En effet, une application web doit passer par son répertoire de gestion du code source, passant par un serveur d'intégration continue et un outil de contrôle qualité jusqu'à son déploiement automatique. Cette dernière phase, basée sur la technologie de virtualisation en conteneurs, transforme ce code source en programme en exécution.

MOTS CLÉS : DevOps, Intégration continue, Déploiement continu, Livraison continue, Automatisation, Virtualisation, Docker, Cloud Computing, Orchestration Conteneurs, …

ABSTRACT:

The objective of this project is to set up a continuous integration platform and continuous delivery and deployment for multi-service web applications. After a study of the main concepts and practices of DevOps, we did out a comparative study of the tools of these practices. Following this comparative study, a needs analysis and a design of the project architecture appeared necessary, before the implementation of this CI / CD pipeline and platform. This project helped us to minimize production time of applications and organize the life cycle of each one. Indeed, now in this project, a web application must go through its source code management directory, passing through a continuous integration server and a quality control tool until its automatic deployment. This last phase of this project, based on container virtualization technology, transforms this source code into a program in execution.

KEY WORDS: DevOps, Continuous integration, Continuous deployment, Continuous delivery, Automation, Virtualization, Docker, Cloud Computing, Container orchestration, …